

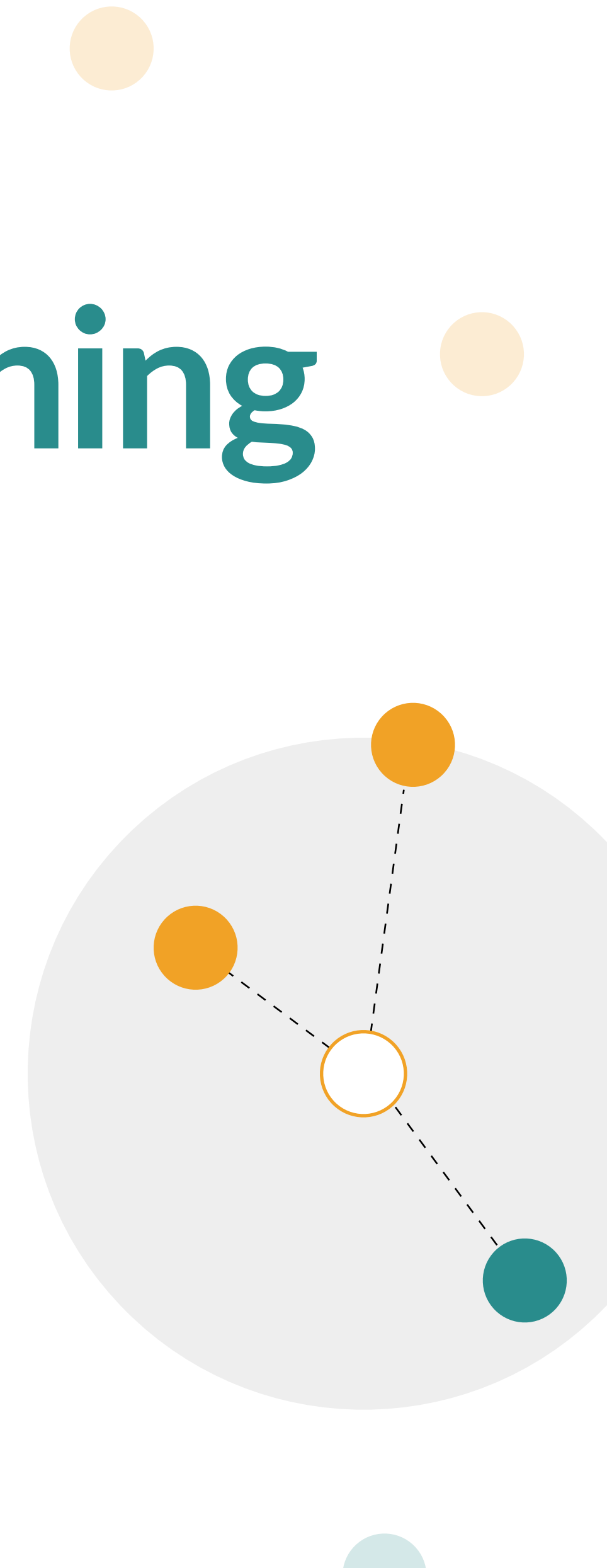
Foundations of Machine Learning

Vito Dichio, Fellow in AI (ENS-PSL)

✉ / vito.dichio@psl.eu

Lecture 2 : Statistical Learning

Master's Degree in **AI & Society** (PSL-PSAI, Fall 2026-27)



Last week

Machine Learning

Anatomy of a Definition

The key is to **find algorithms**—that work, and that we can reason about

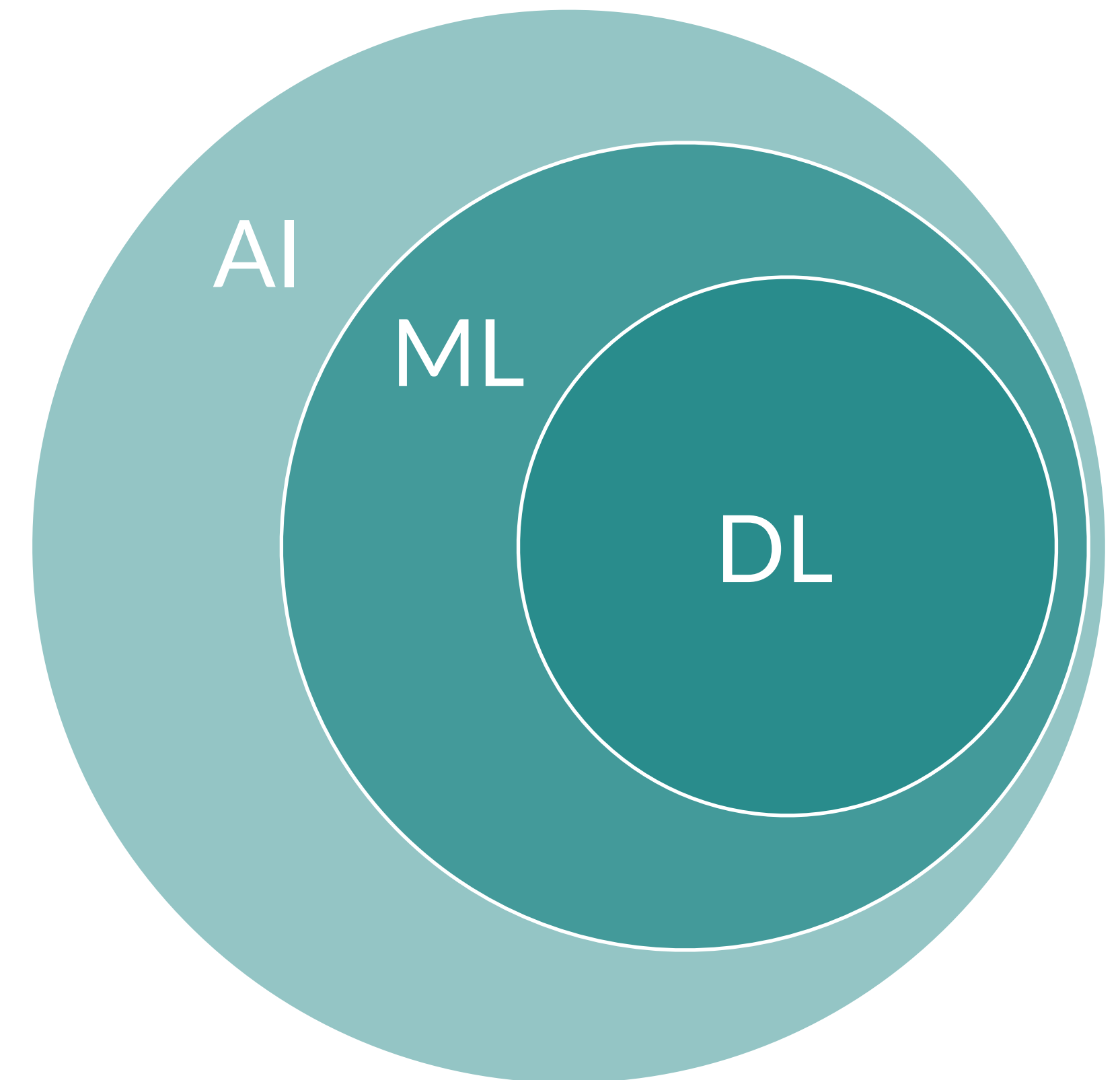
The world is not random, **there is structure** to « discover »



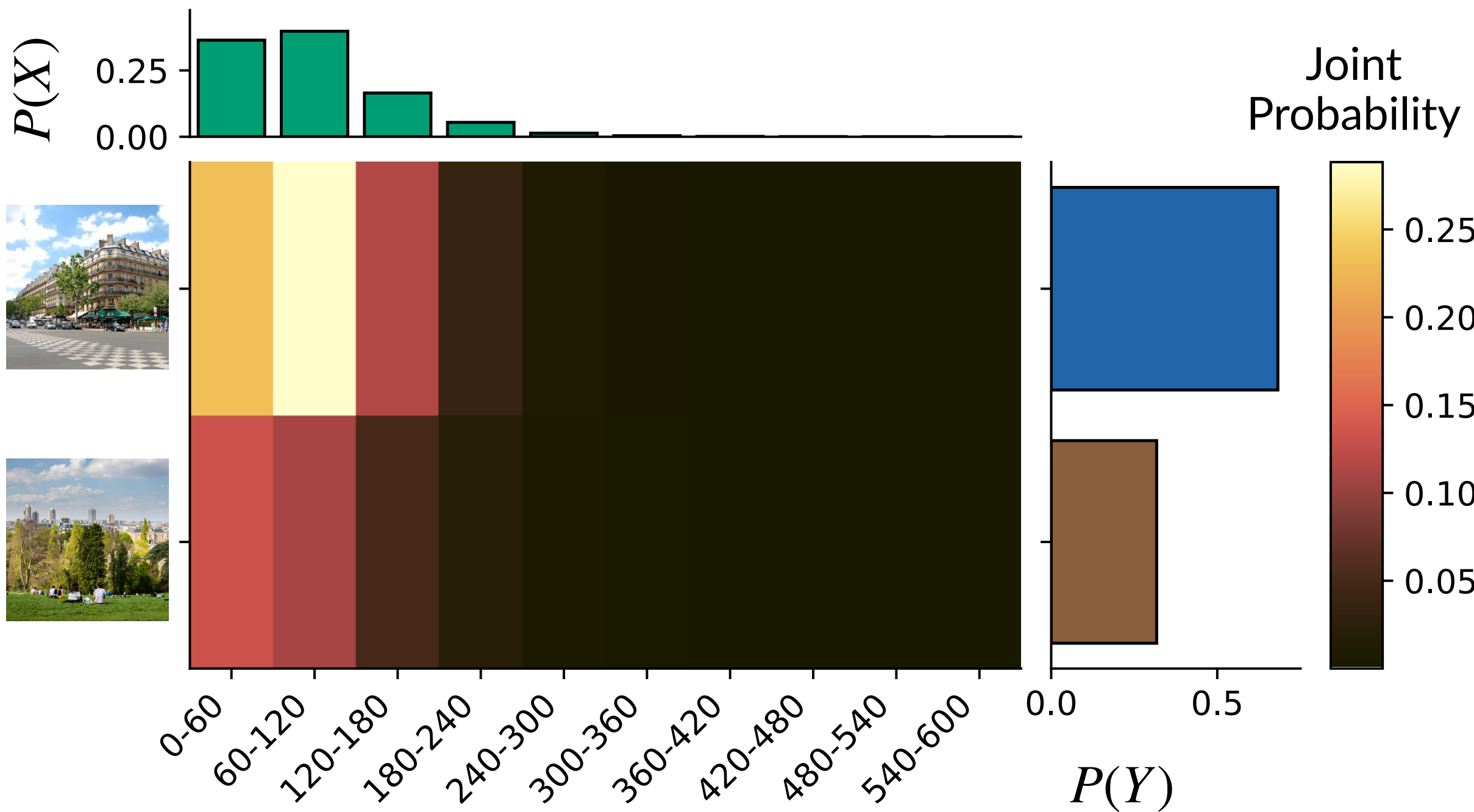
Automatic discovery of regularities in data and their use to take action

Data are **noisy measurements** of the world. The more, the better

This discipline is **goal-oriented**: every algorithm serves a purpose



Probabilities



$n = 220\,022$ trees, from [Paris OpenData](#)

Joint Probability

$$P(X = x_i, Y = y_j) \simeq \frac{n_{ij}}{n}$$

Sum rule

$$P(X) = \sum_Y P(X, Y)$$

Marginal Probability

Product rule

$$P(X, Y) = P(X | Y)P(Y)$$

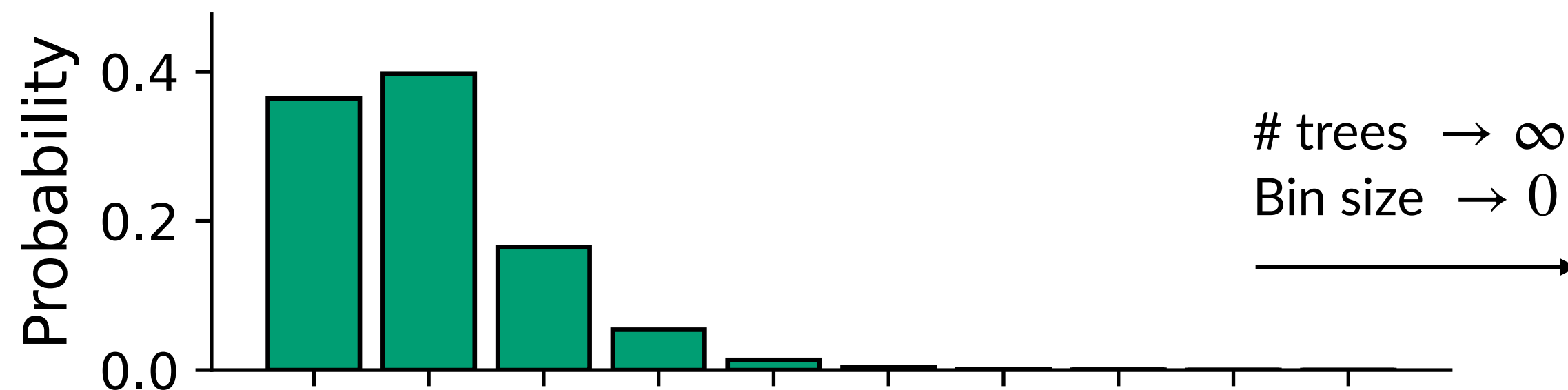
Conditional Probability

Probabilities

A **probability distribution** $P_X : \mathcal{X} \rightarrow [0,1]$ is the complete assignment of a probability to every possible value a random variable $X \in \mathcal{X}$ can take.

For any $f : \mathcal{X} \rightarrow \mathbb{R}$, its **expected value** is

$$\mathbb{E}_X[f(X)] = \sum_{x \in \mathcal{X}} f(x)P(X = x)$$

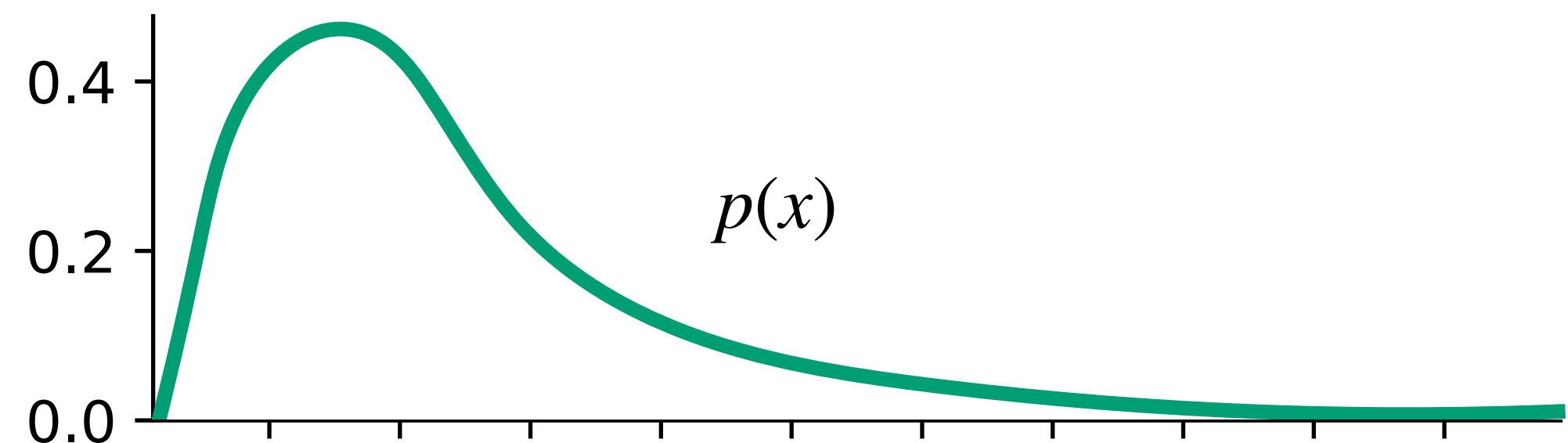


A **probability density** $p(x) : \mathbb{R} \rightarrow [0,\infty)$ is the continuous limit of a distribution. The probability is computed as

$$P(a \leq X \leq b) = \int_a^b p(x) dx$$

For any $f : \mathbb{R} \rightarrow \mathbb{R}$, its **expected value**

$$\mathbb{E}_X[f(X)] = \int f(x)p(x)dx$$



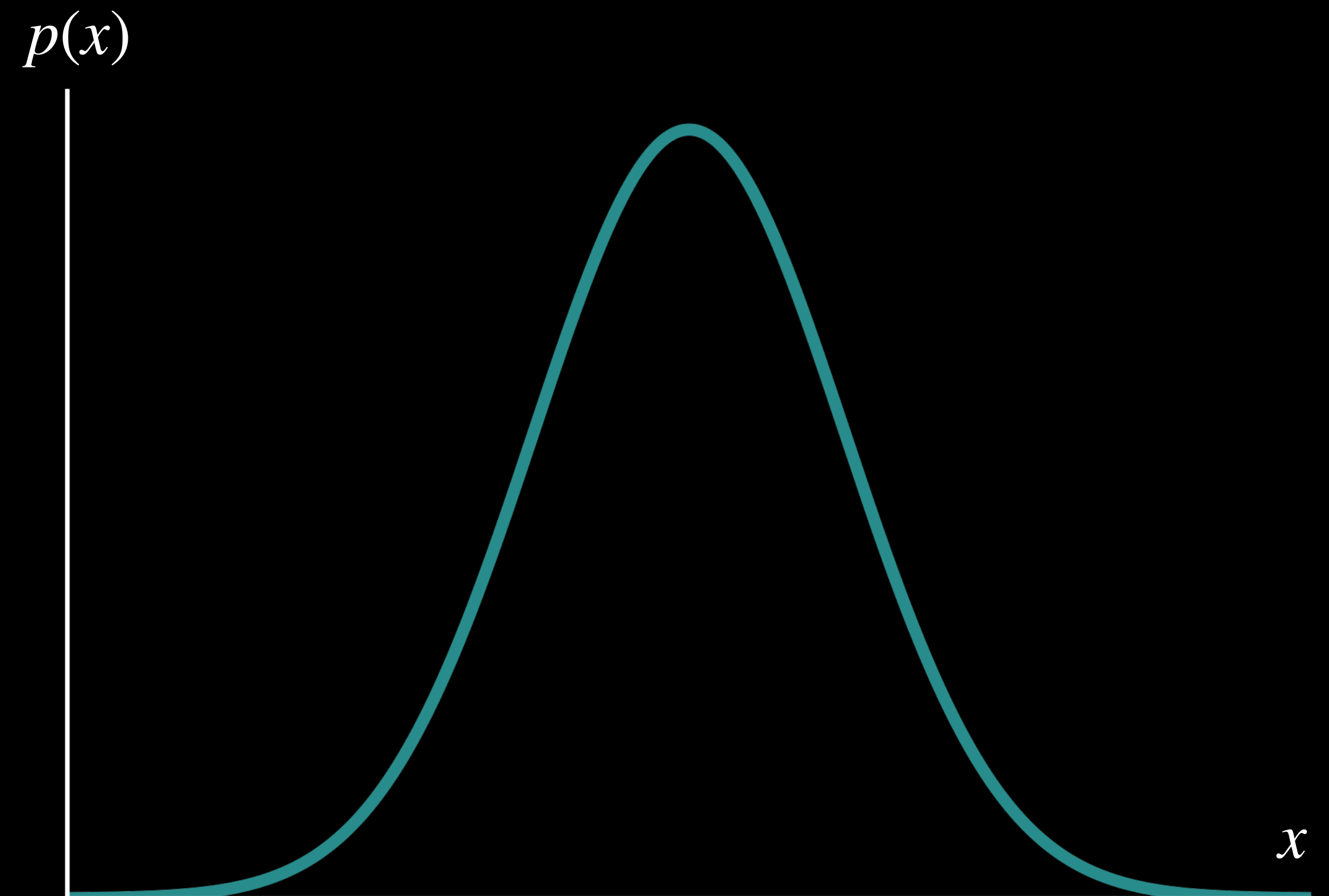
The Gaussian

The Queen of the Densities

- ✓ Where averages end up
- ✓ The most honest distribution, given just mean and variance
- ✓ It stays Gaussian under almost everything
- ✓ It's the default noise model

$$\mathcal{N}(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left\{ -\frac{1}{2\sigma^2}(x - \mu)^2 \right\}$$

$$\mathbb{E}_X[X] = \mu, \quad \text{Var}[X] = \sigma^2$$

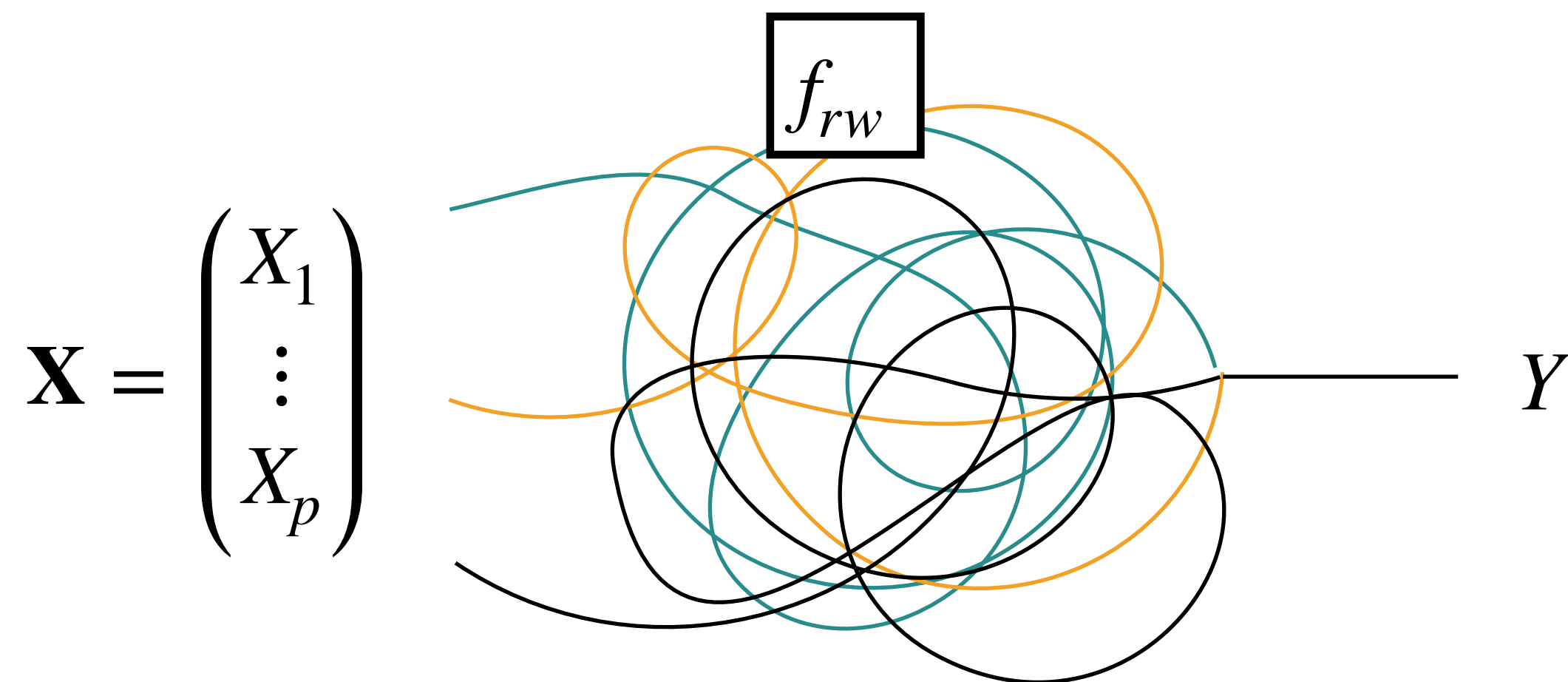


Learning from Data

Statistical Learning Framework



We have a set of variables $\mathbf{X} \in \mathcal{X}$ —the predictors, or features—and a quantity $Y \in \mathcal{Y}$ we want to predict. They are related by some unknown function $f_{rw} : \mathcal{X} \rightarrow \mathcal{Y}$. Our goal is to approximate f_{rw} .



Statistical Learning Framework

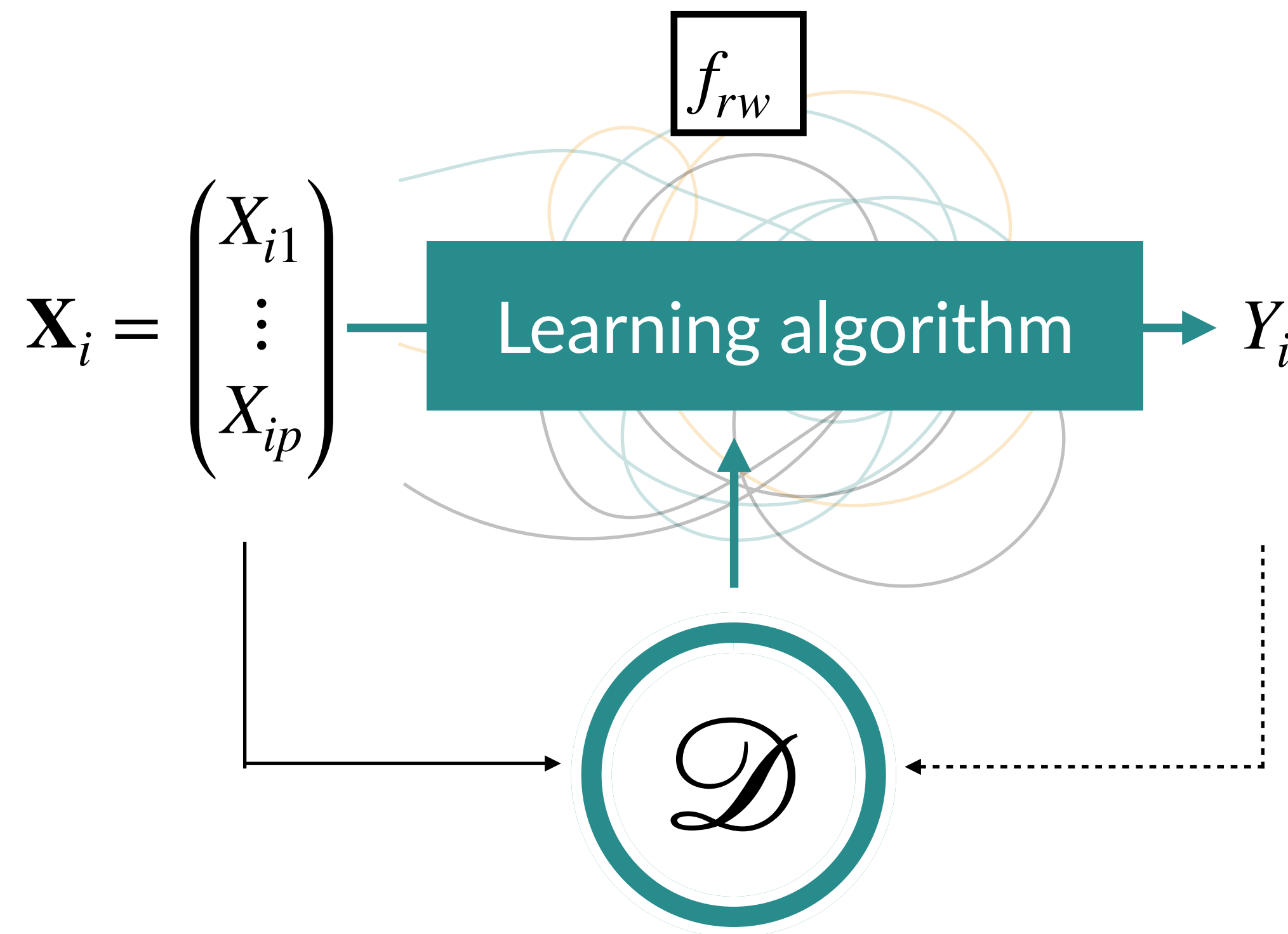


We have a set of variables $\mathbf{X} \in \mathcal{X}$ —the predictors, or features—and a quantity $Y \in \mathcal{Y}$ we want to predict. They are related by some unknown function $f_{rw} : \mathcal{X} \rightarrow \mathcal{Y}$. Our goal is to approximate f_{rw} .



Collect data $\mathcal{D}$ —observations of $\mathbf{X}$ and possibly Y —and **use a learning algorithm** to build an approximation of f_{rw} .

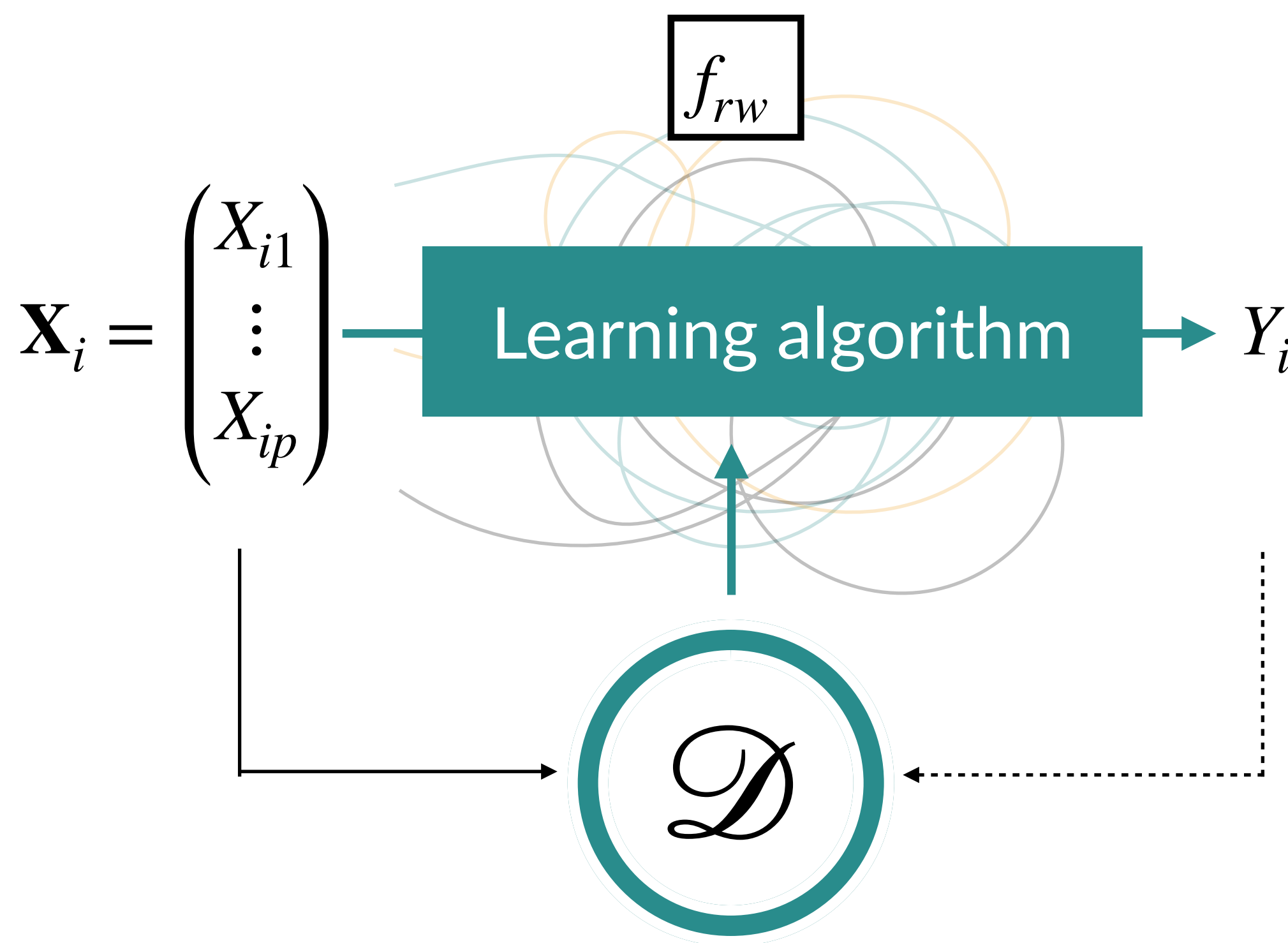
$$\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\} \text{ or } \mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$$



Statistical Learning Framework

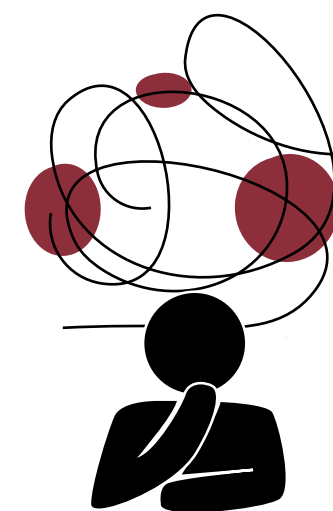


We have a set of variables $\mathbf{X} \in \mathcal{X}$ —the predictors, or features—and a quantity $Y \in \mathcal{Y}$ we want to predict. They are related by some unknown function $f_{rw} : \mathcal{X} \rightarrow \mathcal{Y}$. Our goal is to approximate f_{rw} .



Collect data $\mathcal{D}$ —observations of $\mathbf{X}$ and possibly Y —and **use a learning algorithm** to build an approximation of f_{rw} .

$$\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\} \text{ or } \mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$$



However, we only ever observe **a finite, noisy sample**—so we can never recover the « truth » exactly, only estimate it.

This is why we need statistics: to reason rigorously about an approximation built from limited, imperfect data. Hence, statistical learning.

Kinds of Learning

Supervised

$\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$ —we learn a relationship between predictors and a known response

Examples /

- Can we predict a new outcome accurately?
- Can we understand which factors actually matter?

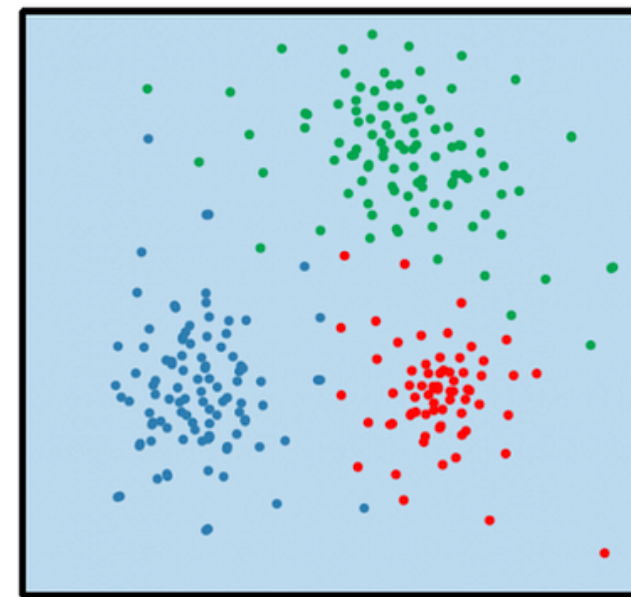
Unsupervised

$\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ —we look for structure in the data itself, with no response to guide us

Examples /

- Do natural groups exist in the data?
- Can we reduce many variables to a few that capture most of the structure?

unsupervised
learning



supervised
learning

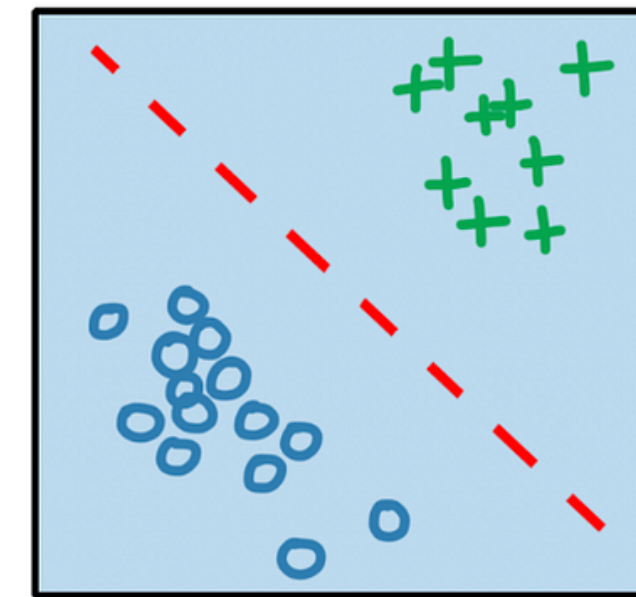


Illustration by [MathWorks](#)

Kinds of Learning

Supervised

$\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$ —we learn a relationship between predictors and a known response

Examples /

- Can we predict a new outcome accurately?
- Can we understand which factors actually matter?

Unsupervised

$\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ —we look for structure in the data itself, with no response to guide us

Examples /

- Do natural groups exist in the data?
- Can we reduce many variables to a few that capture most of the structure?

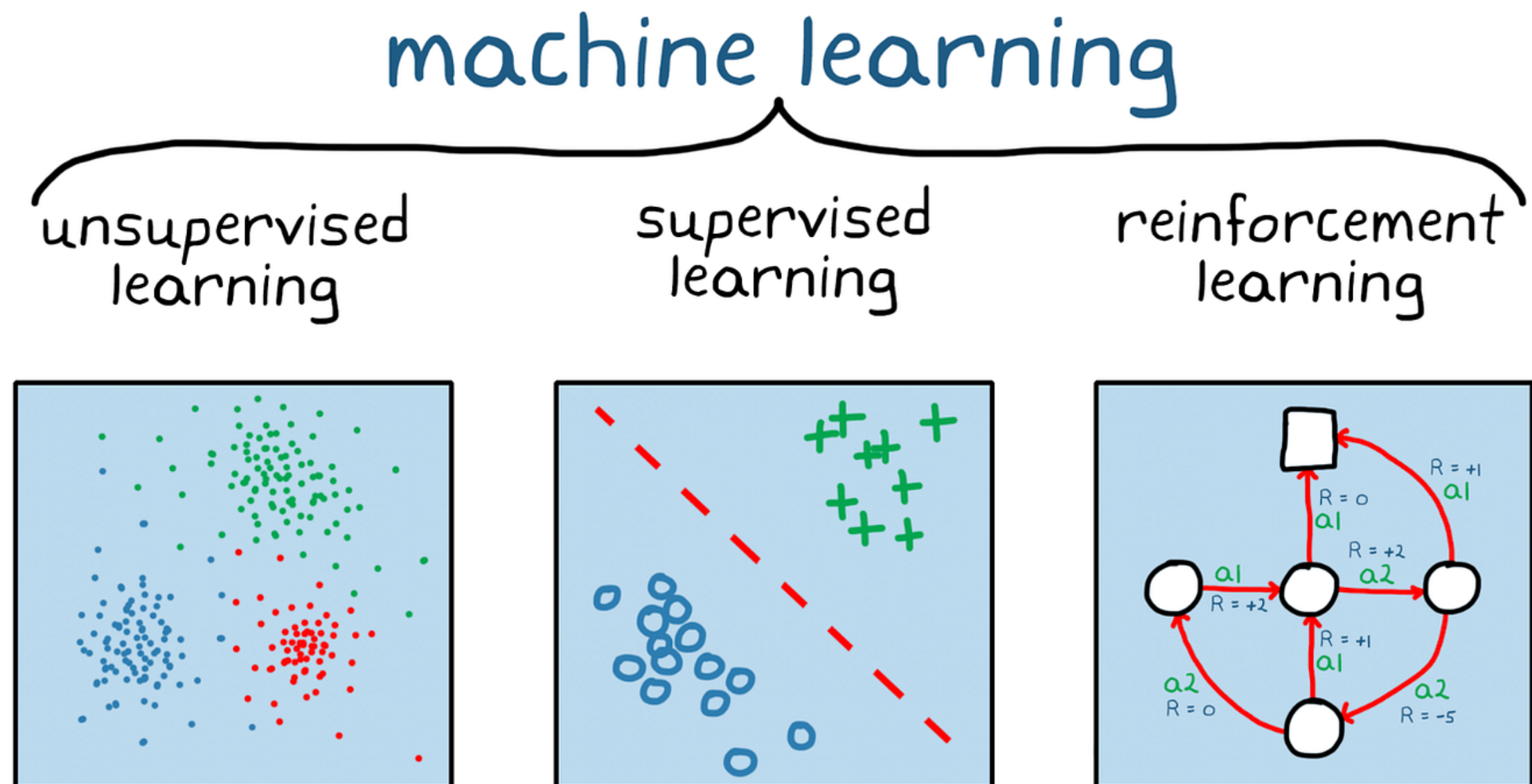


Illustration by [MathWorks](#)

Reinforcement learning : an agent learns to act by interacting with an environment and receiving rewards, rather than from a fixed dataset.
Sutton & Barto, *Reinforcement Learning: An Introduction* (2018)

Not covered in this course 😞

Kinds of Supervised Learning

Regression

Y is **quantitative**, $\mathcal{Y} \subset \mathbb{R}$

Classification

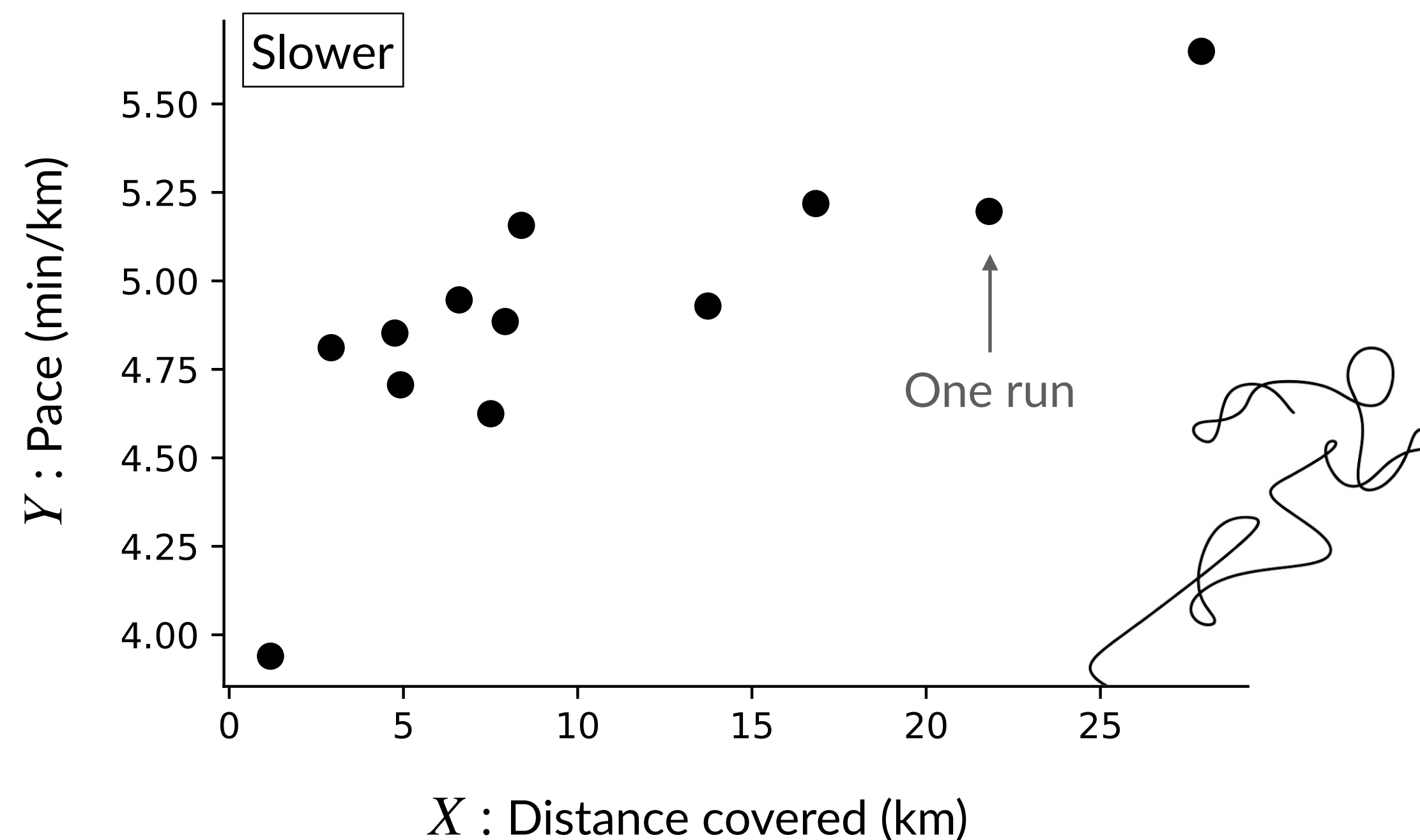
Y is **qualitative**, $\mathcal{Y}$ is a finite set of categories

Kinds of Supervised Learning

Regression

Y is **quantitative**, $\mathcal{Y} \subset \mathbb{R}$

Example / Predicting a runner's pace (min/km)
from the distance covered



Classification

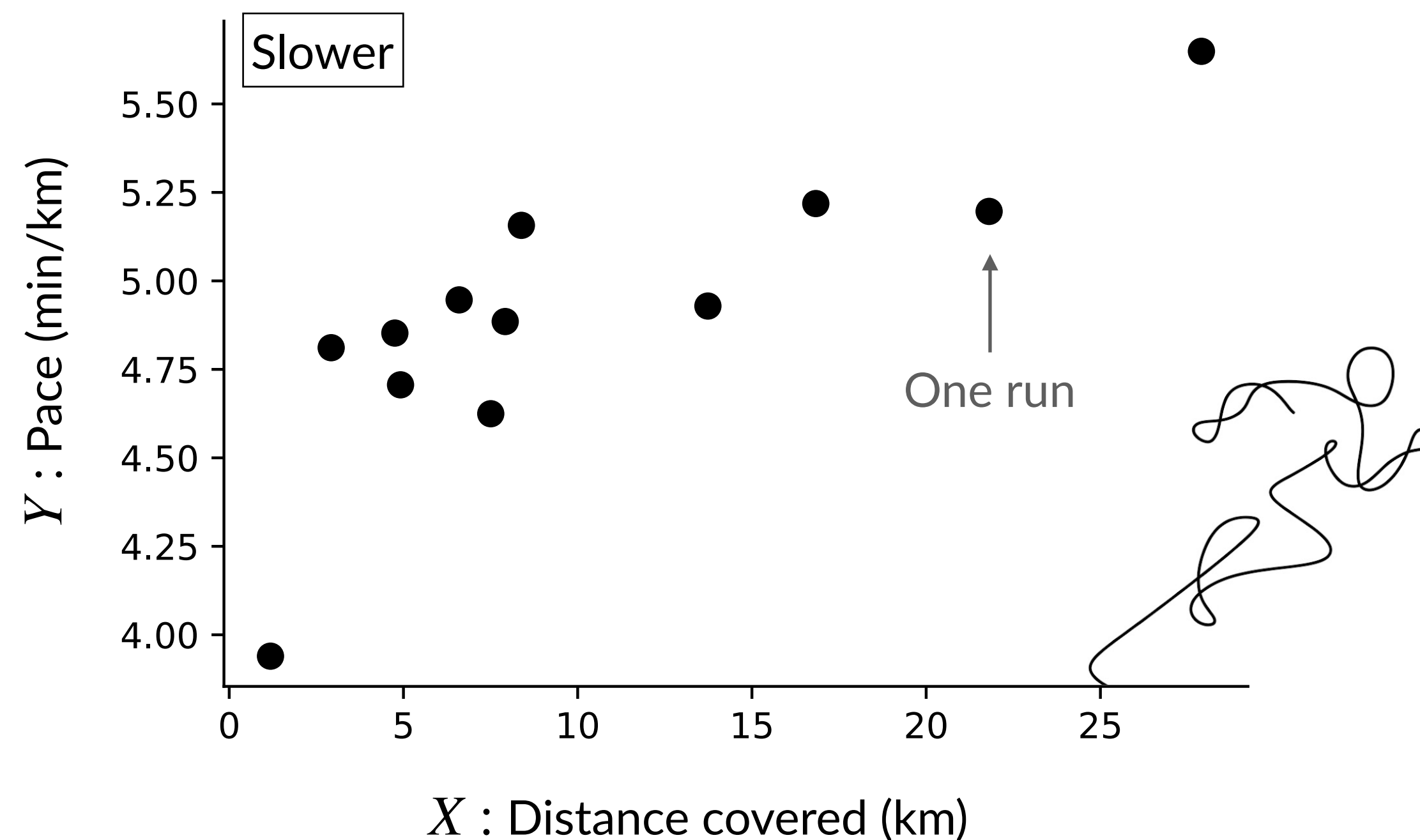
Y is **qualitative**, $\mathcal{Y}$ is a finite set of categorie

Kinds of Supervised Learning

Regression

Y is **quantitative**, $\mathcal{Y} \subset \mathbb{R}$

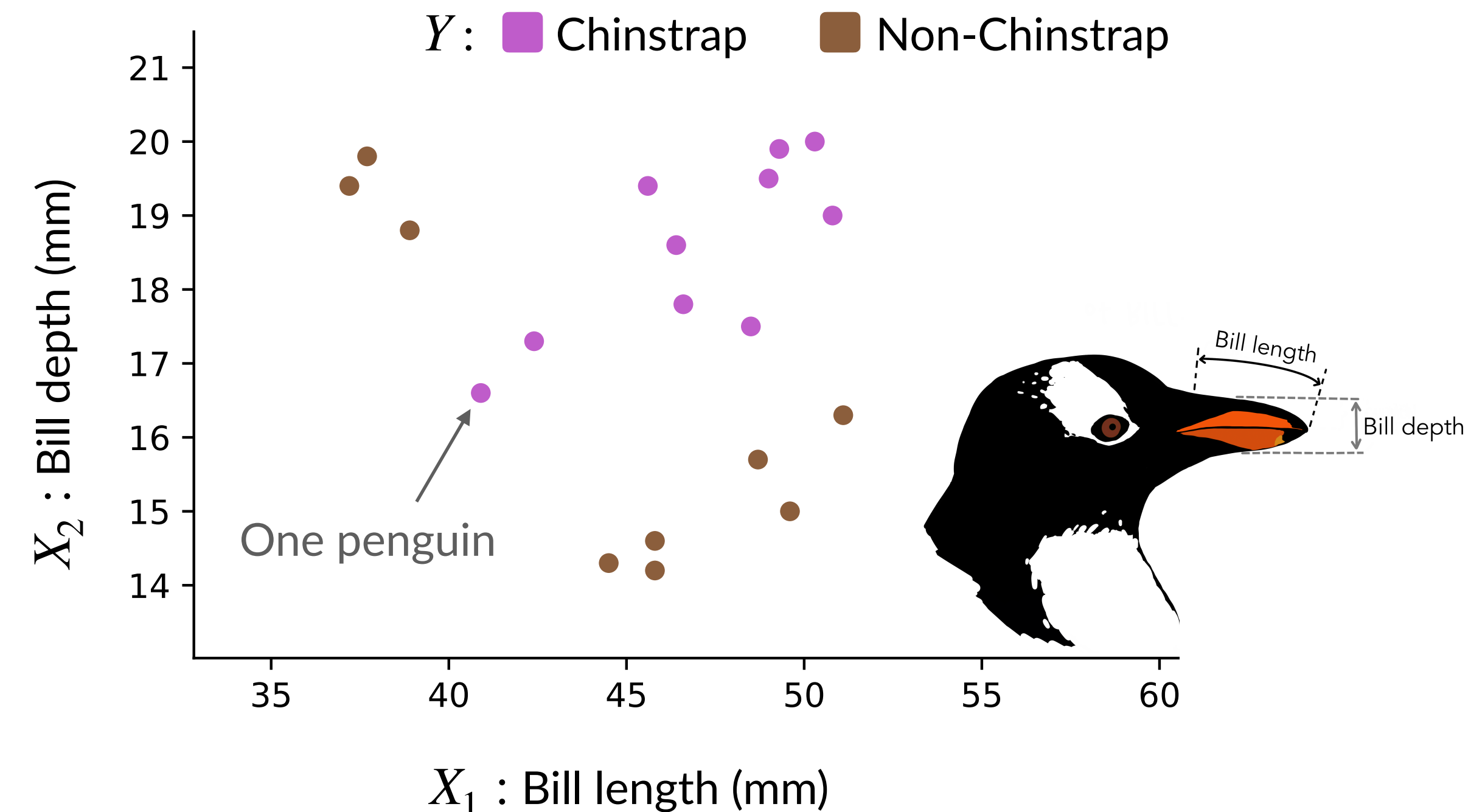
Example / Predicting a runner's pace (min/km) from the distance covered



Classification

Y is **qualitative**, $\mathcal{Y}$ is a finite set of categorie

Example / Predicting whether a penguin belongs to the species « Chinstrap » from its bill length and depth



Regression

A Reasonable Model

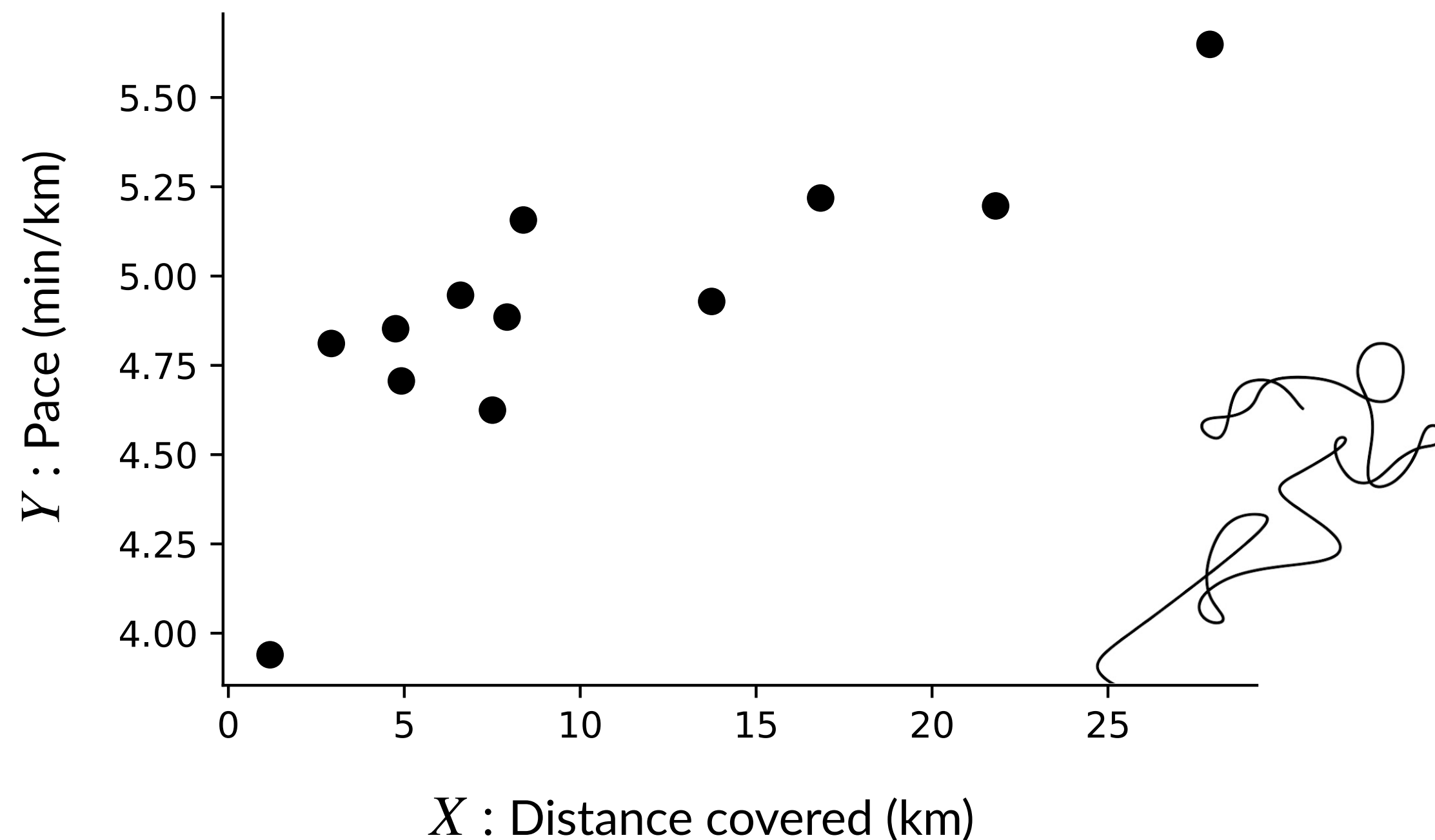
Structure for the Unknown

Assumption / We model Y and X as:

$$Y = f_{rw}(X) \simeq f(X) + \varepsilon$$

The systematic information
that X provides about Y .

This we can learn



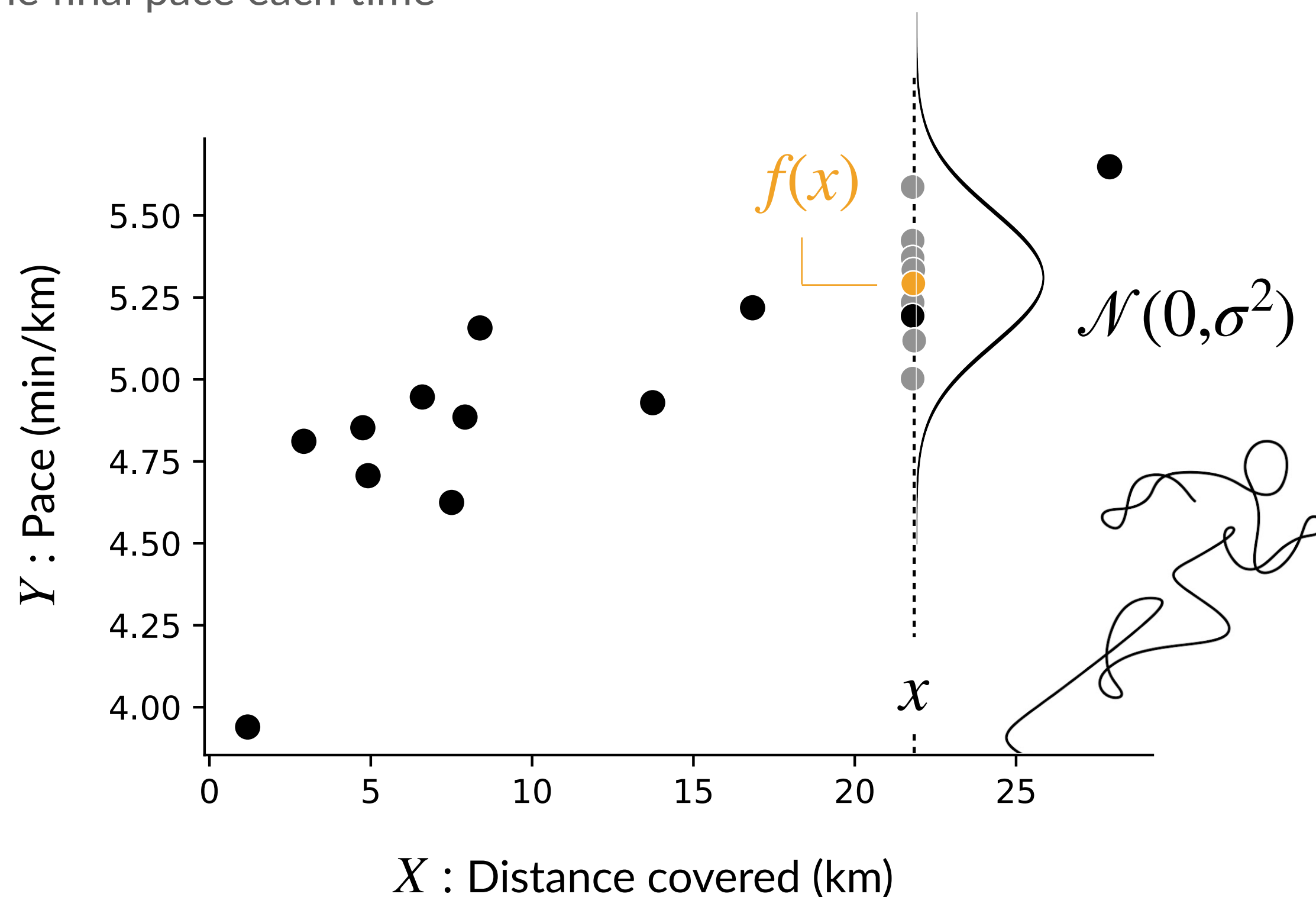
$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

Everything else, independent
of X . **This we can't learn**—a
hard floor on how well we can
ever predict

A Reasonable Model

Structure for the Unknown

Running many times the exact same distance x and recording the final pace each time



Assumption / We model Y and X as:

$$Y = f_{rw}(X) \simeq f(X) + \varepsilon$$

The systematic information
that X provides about Y .

This we can learn

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

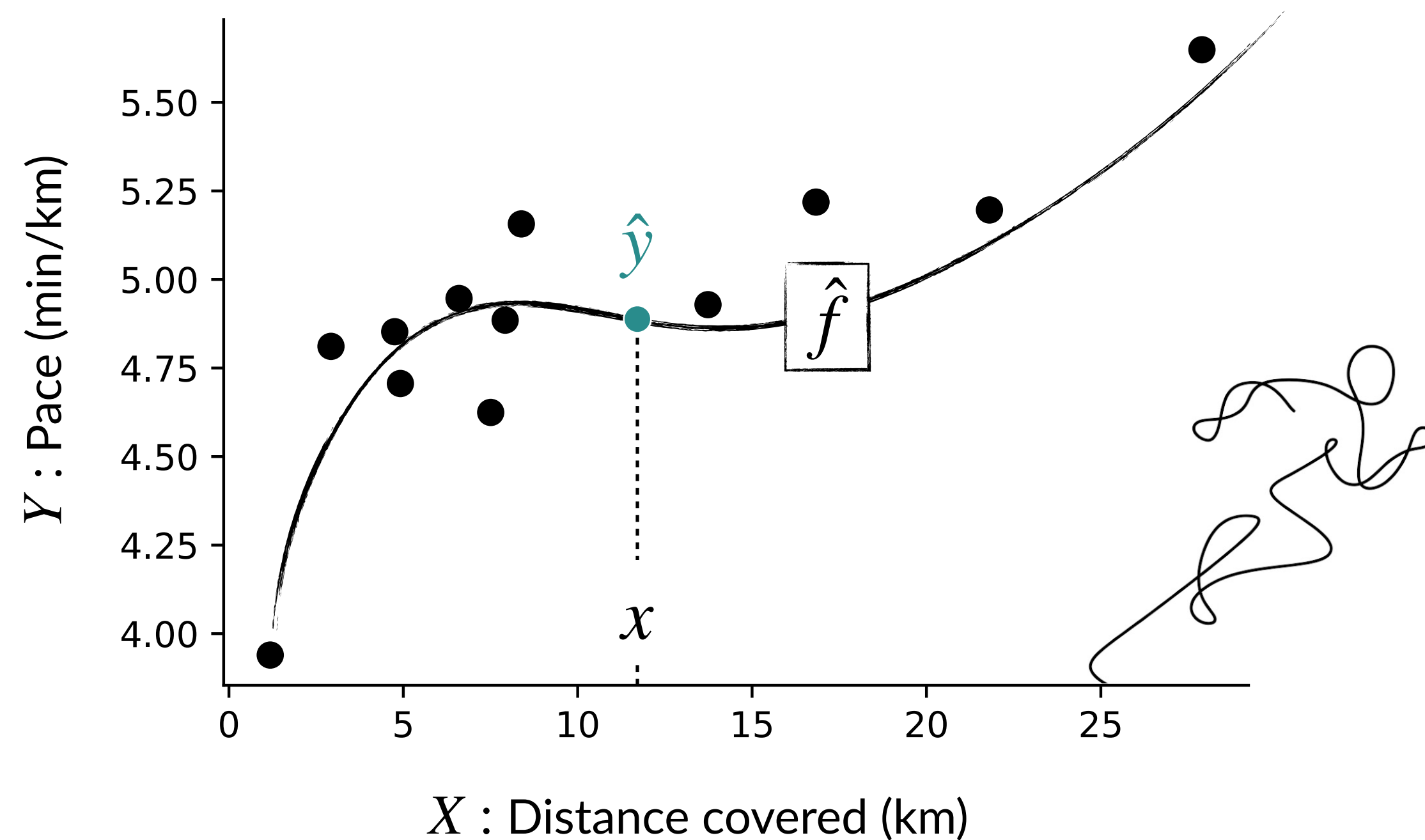
Everything else, independent
of X . **This we can't learn**—a
hard floor on how well we can
ever predict

Note that, $\mathbb{E}_\varepsilon[Y|X=x] = f(x)$. In words, imagine observing y many times at the same x —each time slightly different, due to ε . $f(x)$ is what those values average to.

What for?

Suppose we had an estimate $\hat{f}$ of f . Given a new value of X , we can then form a **prediction**:

$$\hat{Y} = \hat{f}(X)$$

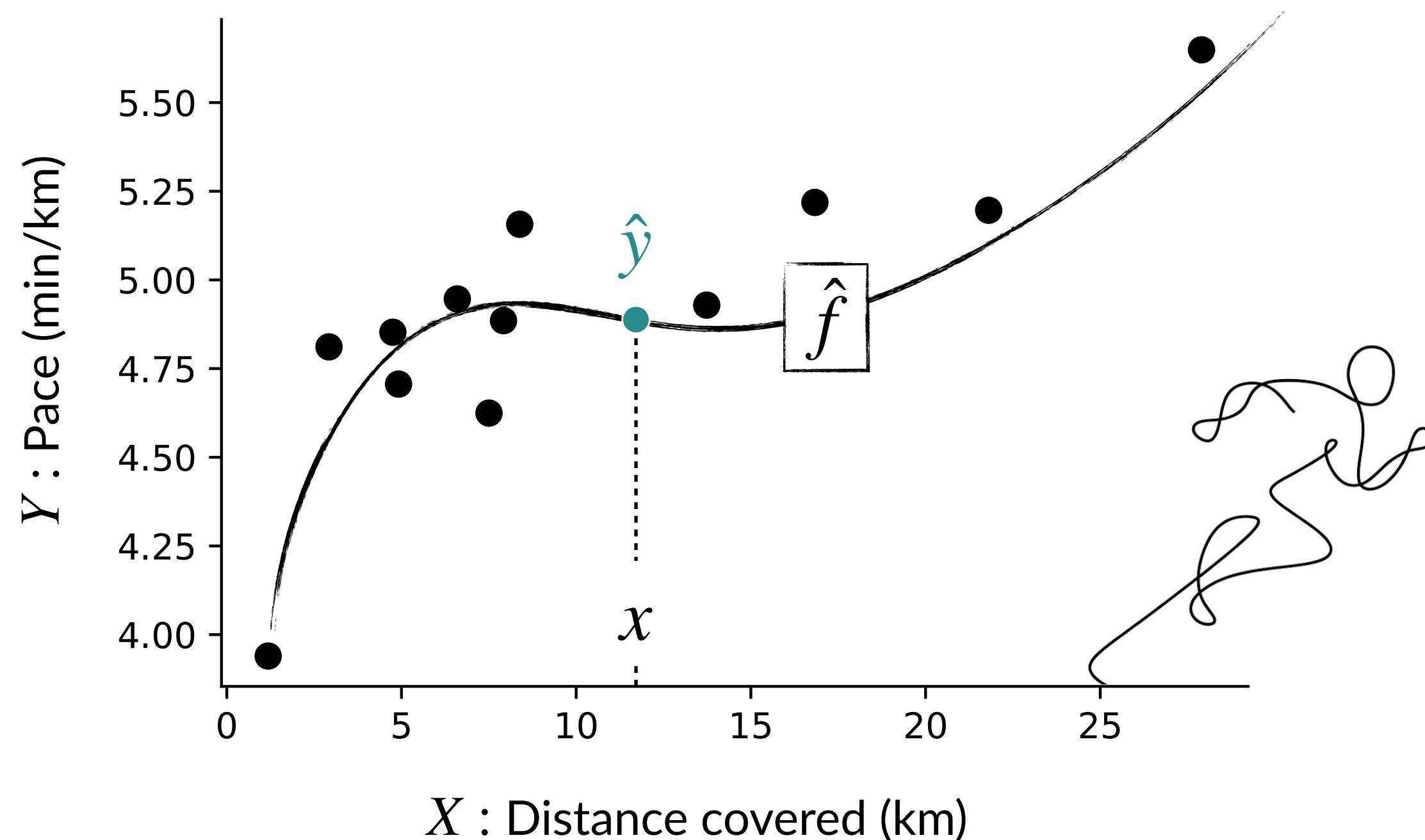


What for?

Prediction vs Inference

Suppose we had an estimate $\hat{f}$ of f . Given a new value of X , we can then form a **prediction**:

$$\hat{Y} = \hat{f}(X)$$



Prediction

$\hat{f}$ is treated as a black-box. What matters is **achieving the best accuracy**.

Ex. / what pace should we expect for a new distance, say 12km?

Inference

Understand how Y depends on X . What matters is **interpretability**—the shape of $\hat{f}$ itself.

Ex. / How does pace change with distance, and what does that tell us about endurance?

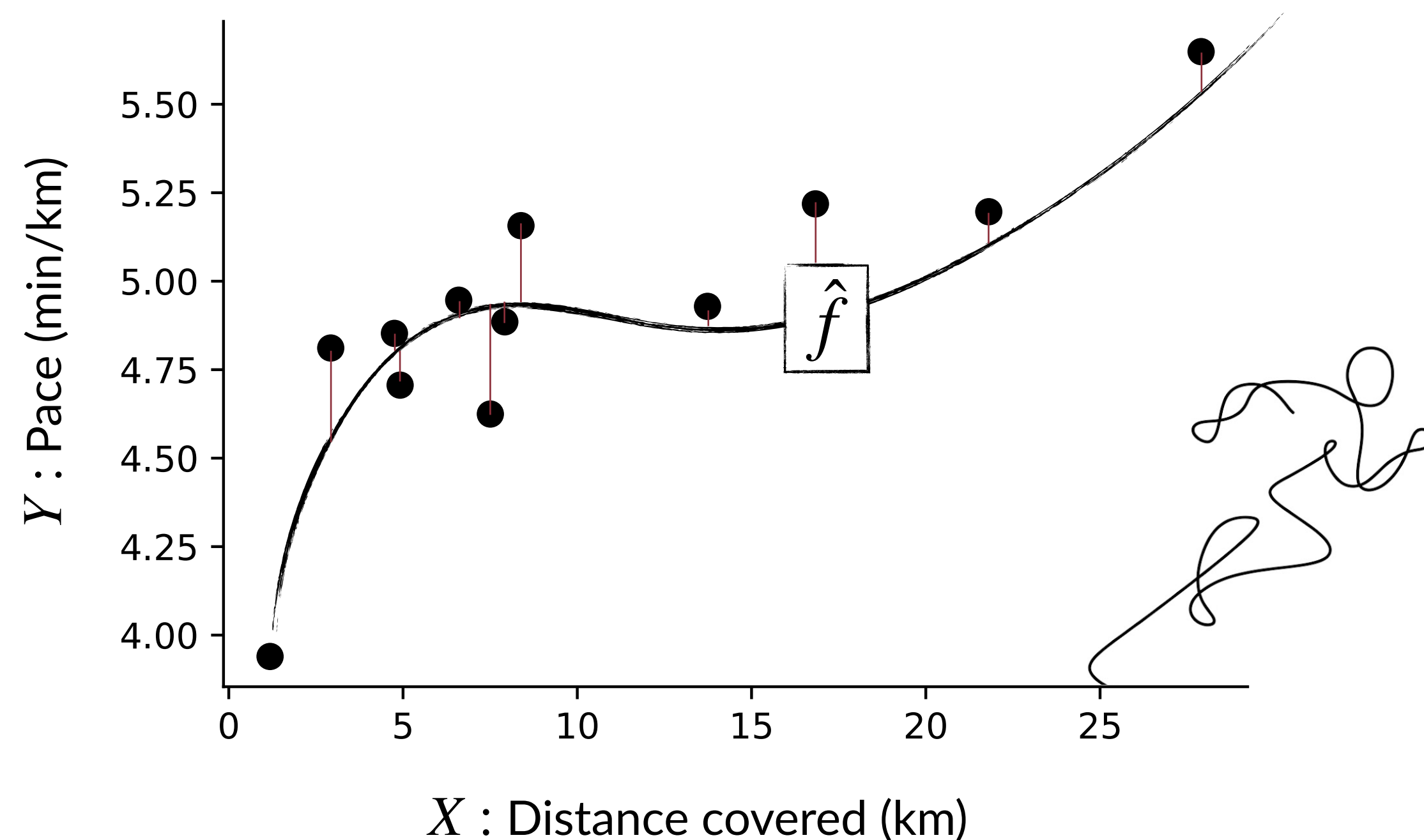


To Predict
or to Infer?

How Good is $\hat{f}$?

Measuring the Quality of Fit

We need a precise, quantitative way to measure how good the predictions are—a number we can compute, compare and optimise. This is the role of the **error**, or **loss**, function.



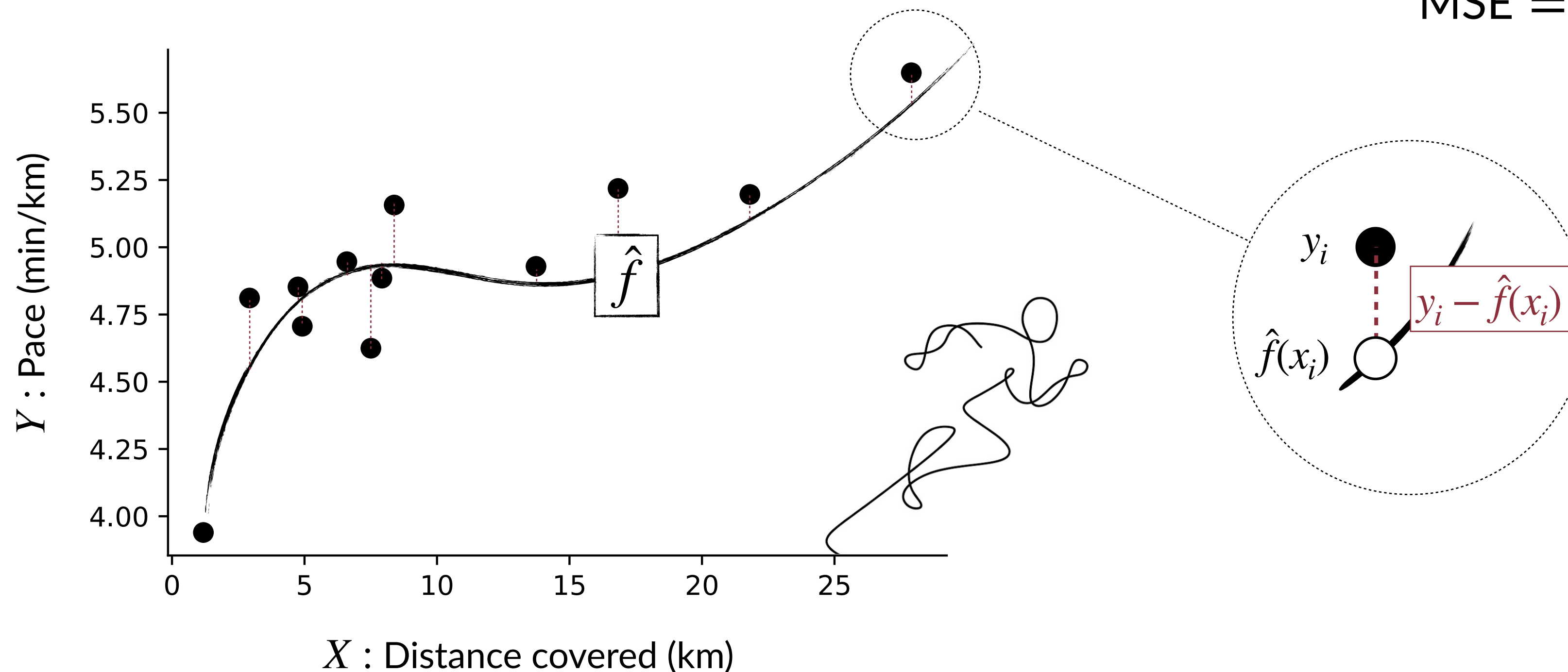
How Good is $\hat{f}$?

Measuring the Quality of Fit

We need a precise, quantitative way to measure how good the predictions are—a number we can compute, compare and optimise. This is the role of the **error**, or **loss**, function.

In regression problems, a standard choice is the **Mean Squared Error (MSE)**

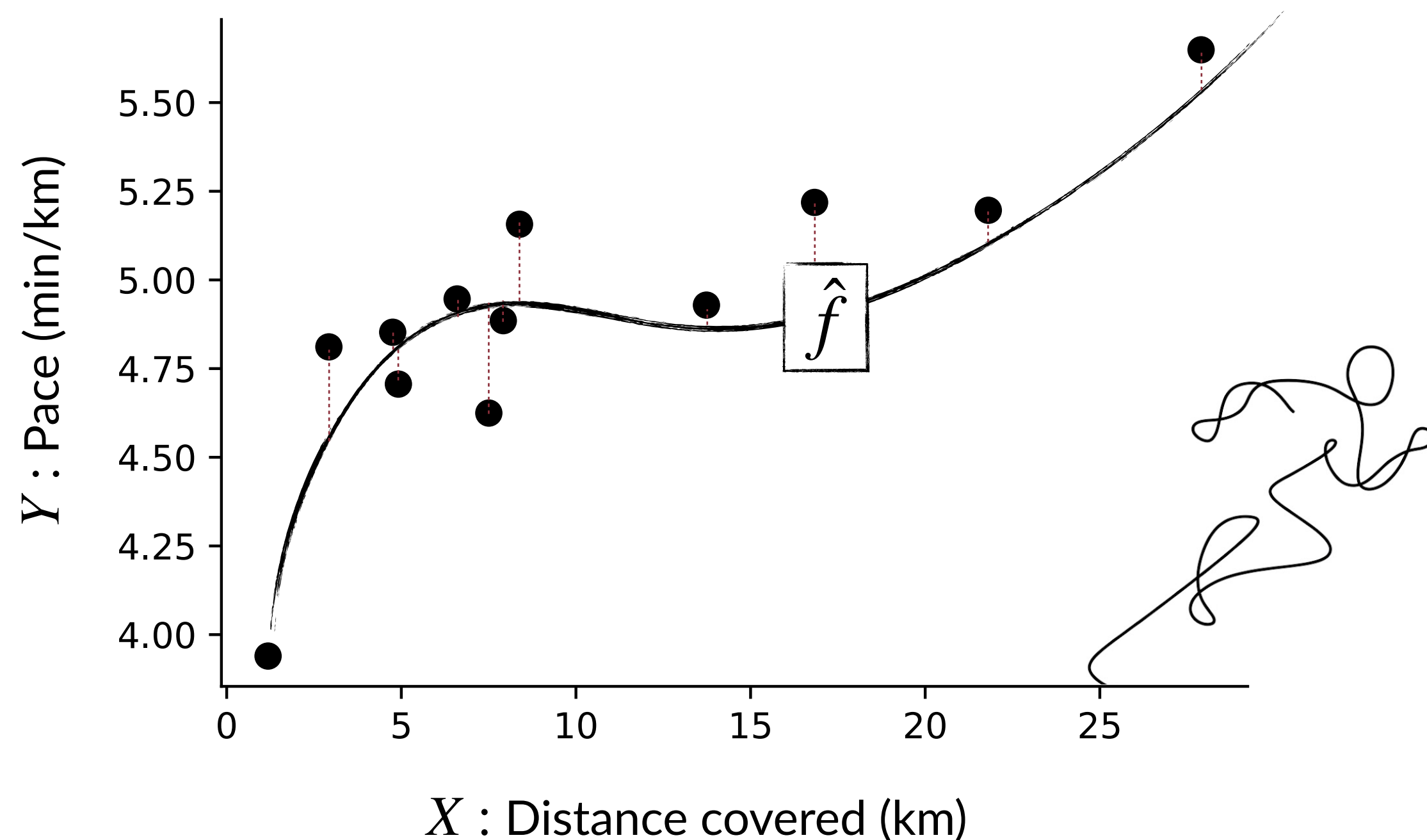
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{f}(x_i) \right)^2$$



How Good is $\hat{f}$?

Measuring the Quality of Fit

We need a precise, quantitative way to measure how good the predictions are—a number we can compute, compare and optimise. This is the role of the **error**, or **loss**, function.



In regression problems, a standard choice is the **Mean Squared Error (MSE)**

$$\text{MSE} = \frac{1}{n} \sum_{i=0}^n \left(y_i - \hat{f}(x_i) \right)^2$$

A tentative recipe—Empirical Risk Minimisation

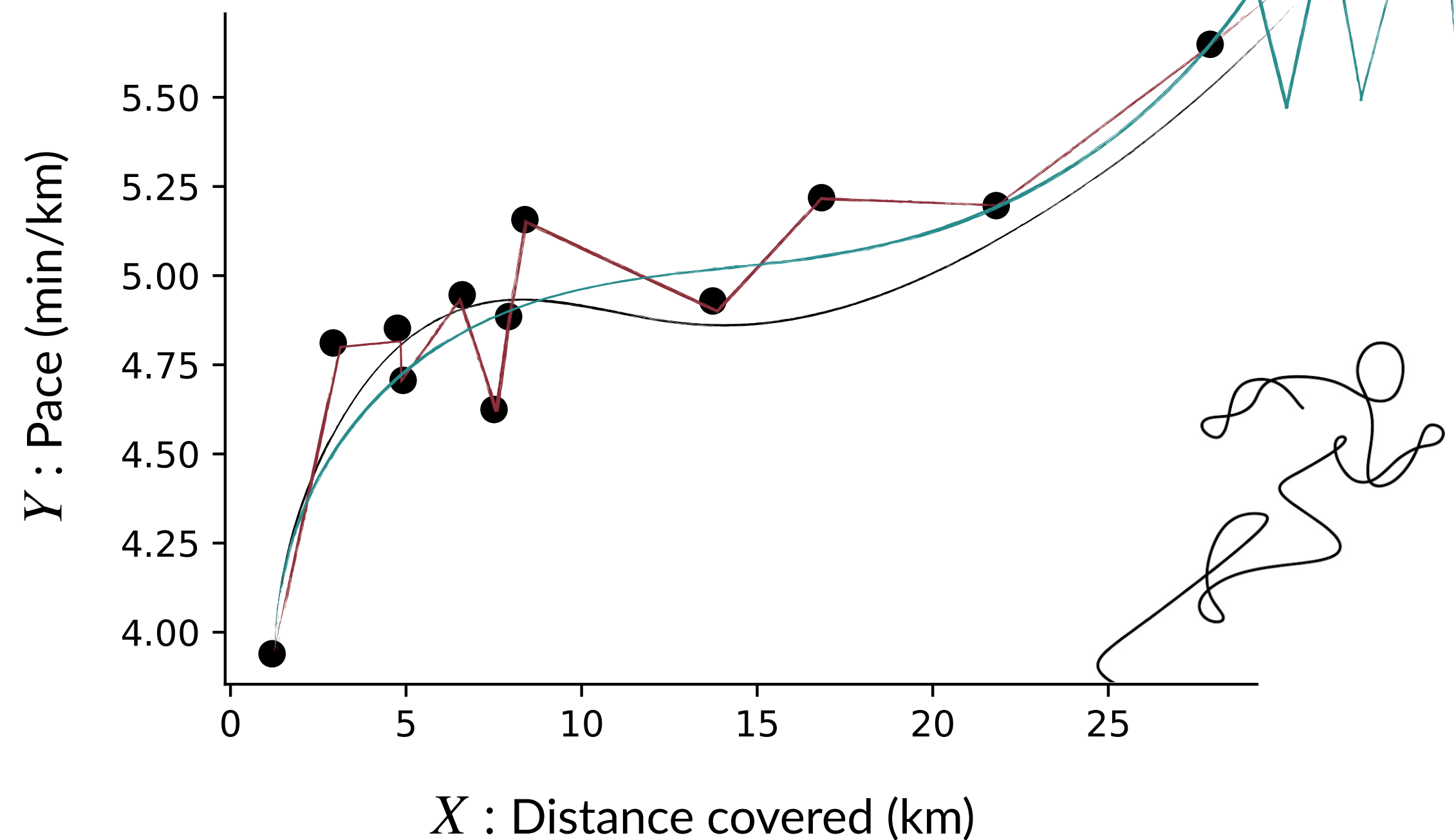
Find the function $\hat{f}$ that makes the MSE as small as possible across our data:

$$\hat{f} = \arg \min_f \frac{1}{n} \sum_{i=1}^n \left(y_i - f(x_i) \right)^2$$

Searching for $\hat{f}$

Estimating f means **searching over possible functions** for an optimal $\hat{f}$

The space of possible functions is unconstrained—infininitely many fit any finite dataset perfectly, most of them wild and meaningless. We need to restrict where we look

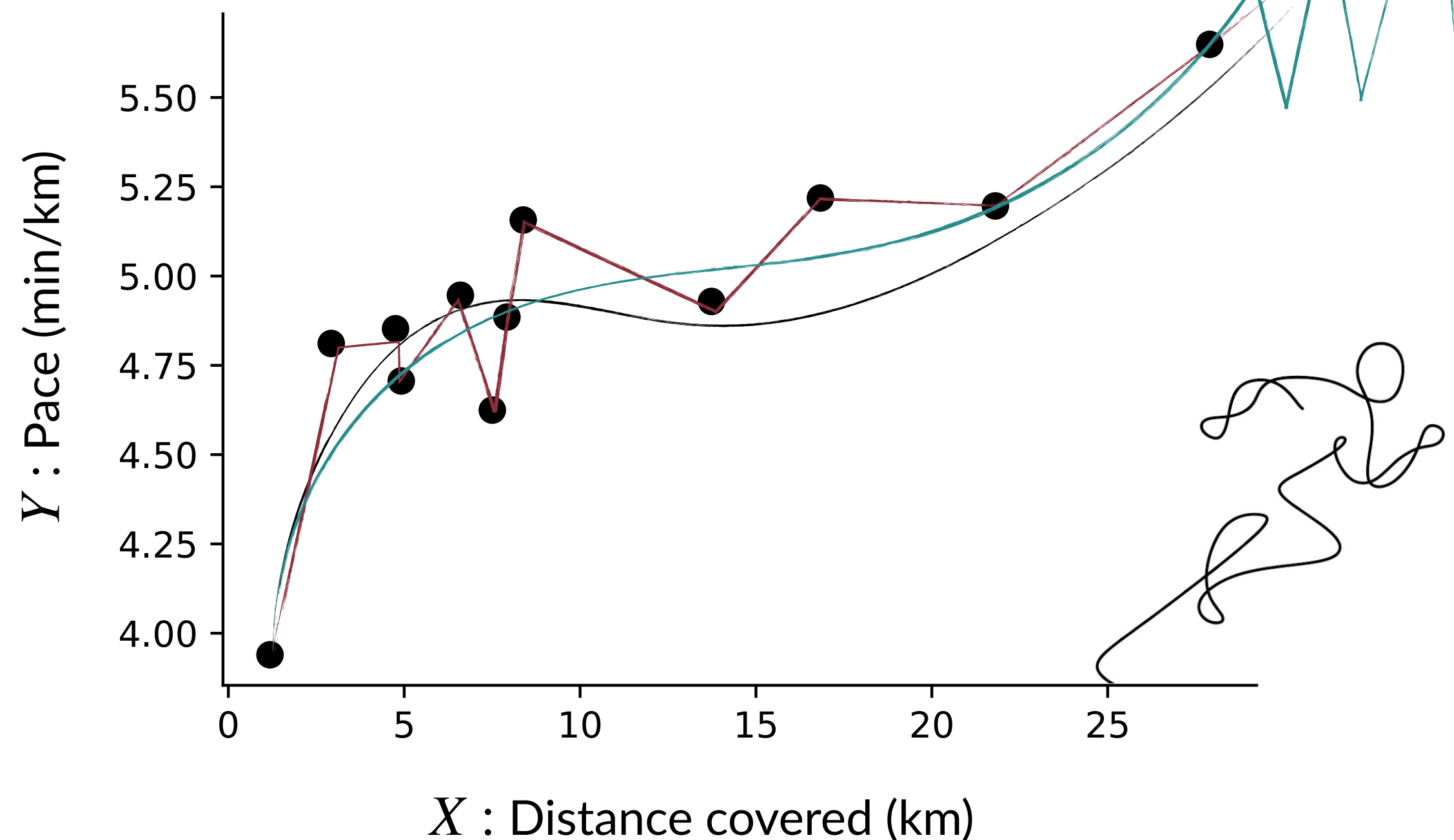


Searching for $\hat{f}$

Parametric vs Non-parametric

Estimating f means **searching over possible functions** for an optimal $\hat{f}$

The space of possible functions is unconstrained—infininitely many fit any finite dataset perfectly, most of them wild and meaningless. We need to restrict where we look



Parametric

Choose a **family of functions** f_β , **indexed**—or « parametrised »—**by a set of parameters** $\beta \in \mathbb{R}^{p+1}$.

→ Estimating $\hat{f}$ now means searching the optimal set of parameters in a fixed and tractable space with $p+1$ coordinates

Non-parametric

Don't commit to any fixed family. Instead, restrict $\hat{f}$ only through **general constraints**, leaving the data free to shape its form

→ Estimating $\hat{f}$ means searching directly over functions satisfying these constraints

The constraints are typically about how wild $\hat{f}$ is allowed to be—how much it can bend or vary, or how sensitive it is to individual data points

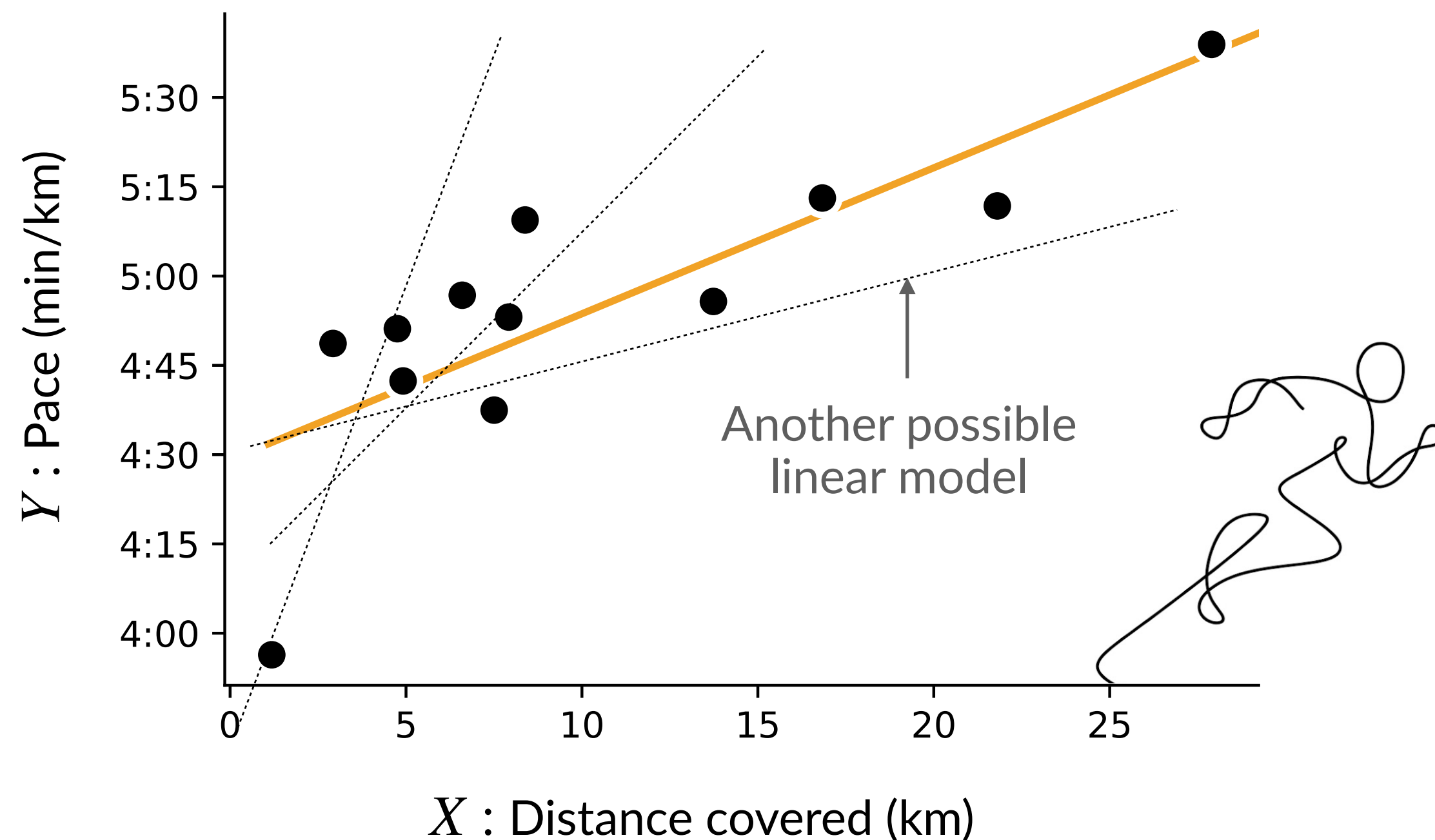
Modelling Pace

A Linear Attempt

The optimal solutions shown here were found by Empirical Risk Minimisation—the mechanics of *how* are left for later lectures

Optimal fit

$$\hat{\beta}_0 = 4:29 \text{ min/km}, \quad \hat{\beta}_1 = +2.45 \text{ s/km}^2$$



Linear Model

The **simplest** non-trivial **parametric** choice: assume pace depends on distance linearly

$$f_{\beta}(X) = \beta_0 + \beta_1 X$$

Pros /

Interpretable : β_1 has a direct meaning—the change in pace per extra km. Ideal for inference

Stable : with only two parameters, the fit doesn't change much from one sample to another

Cheap : only two parameters to fit

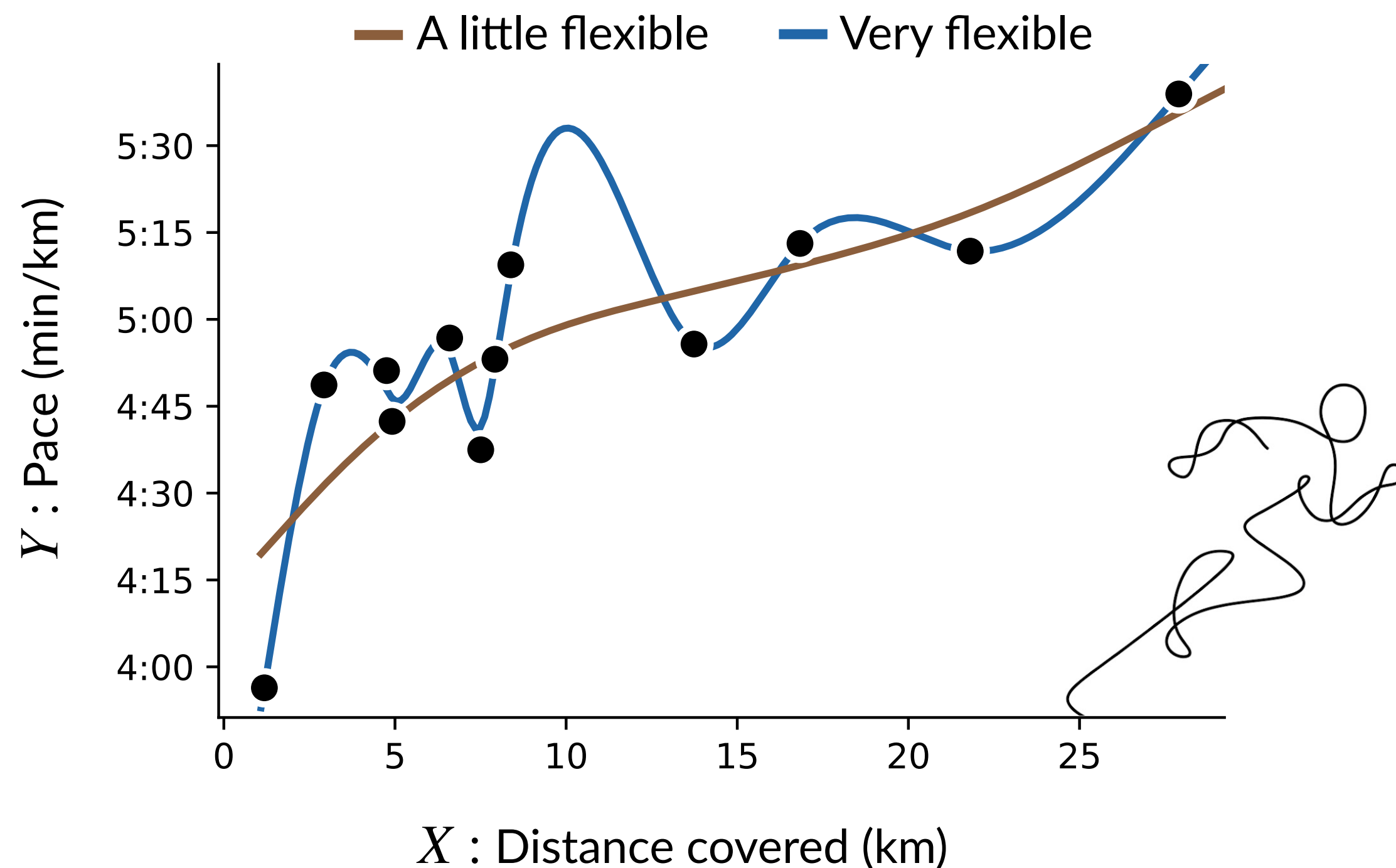
Cons /

Rigid : the model relies on a strong linear assumption. If the true pattern curves, bends, or levels off, every line will miss it the same way, everywhere

Modelling Pace

A Flexible Attempt

The optimal solutions shown here were found by Empirical Risk Minimisation—the mechanics of *how* are left for later lectures



Smoothing Splines

A **non-parametric method**: the curve bends to follow the data directly. How much it's allowed to bend is controlled by a **hyperparameter**

A number we choose before fitting

Pros /

Flexible: not locked into one shape in advance—can bend to follow patterns

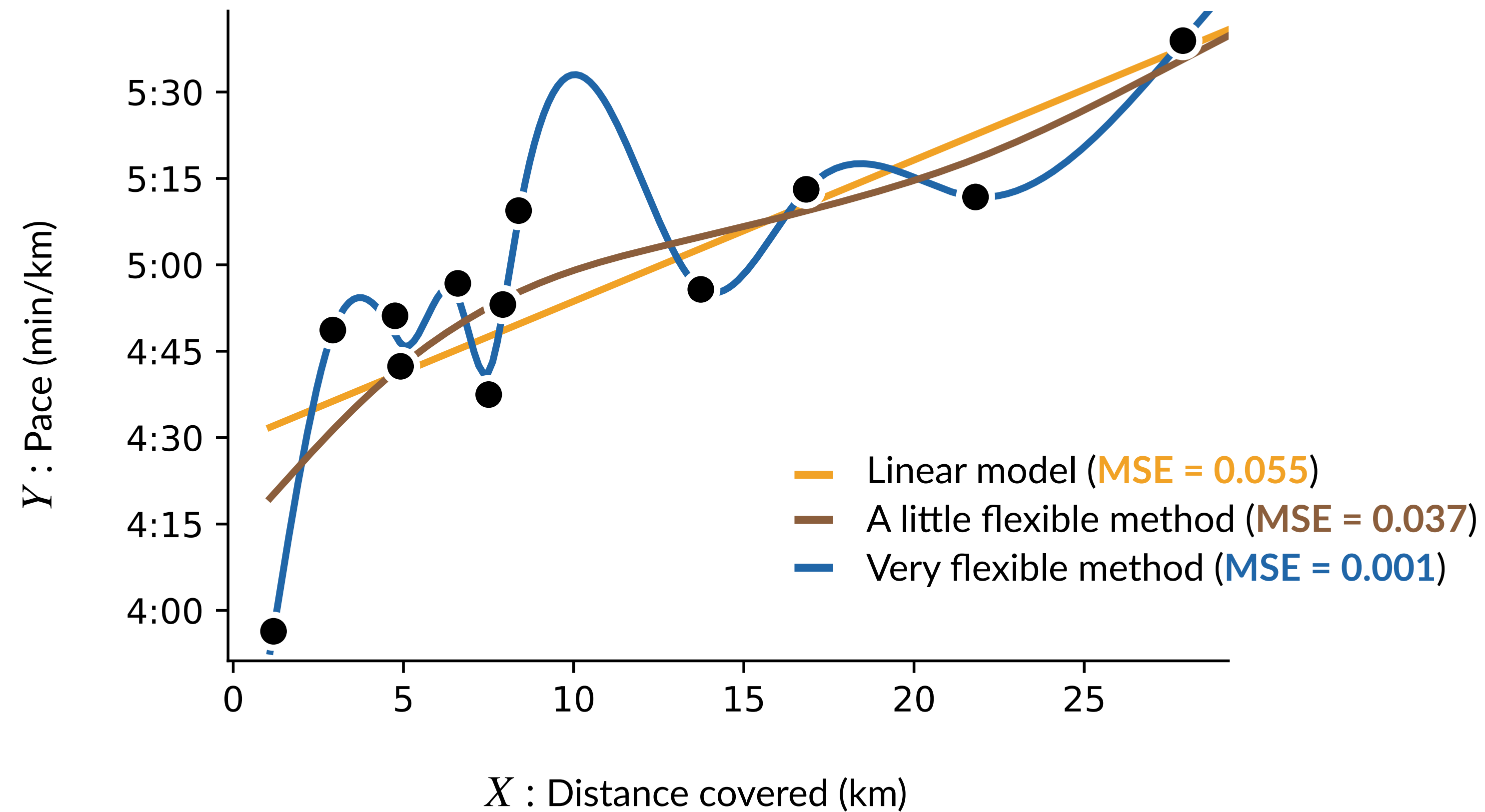
Cons /

Needs a choice: the level of « flexibility » has to be set somehow. If too rigid, it may miss real patterns in the data (as the linear model). If too flexible, it may end up capturing quirks specific to this sample.

Limited interpretability: there is no single coefficient summarising "the effect of distance"—only the shape of the fitted curve itself.

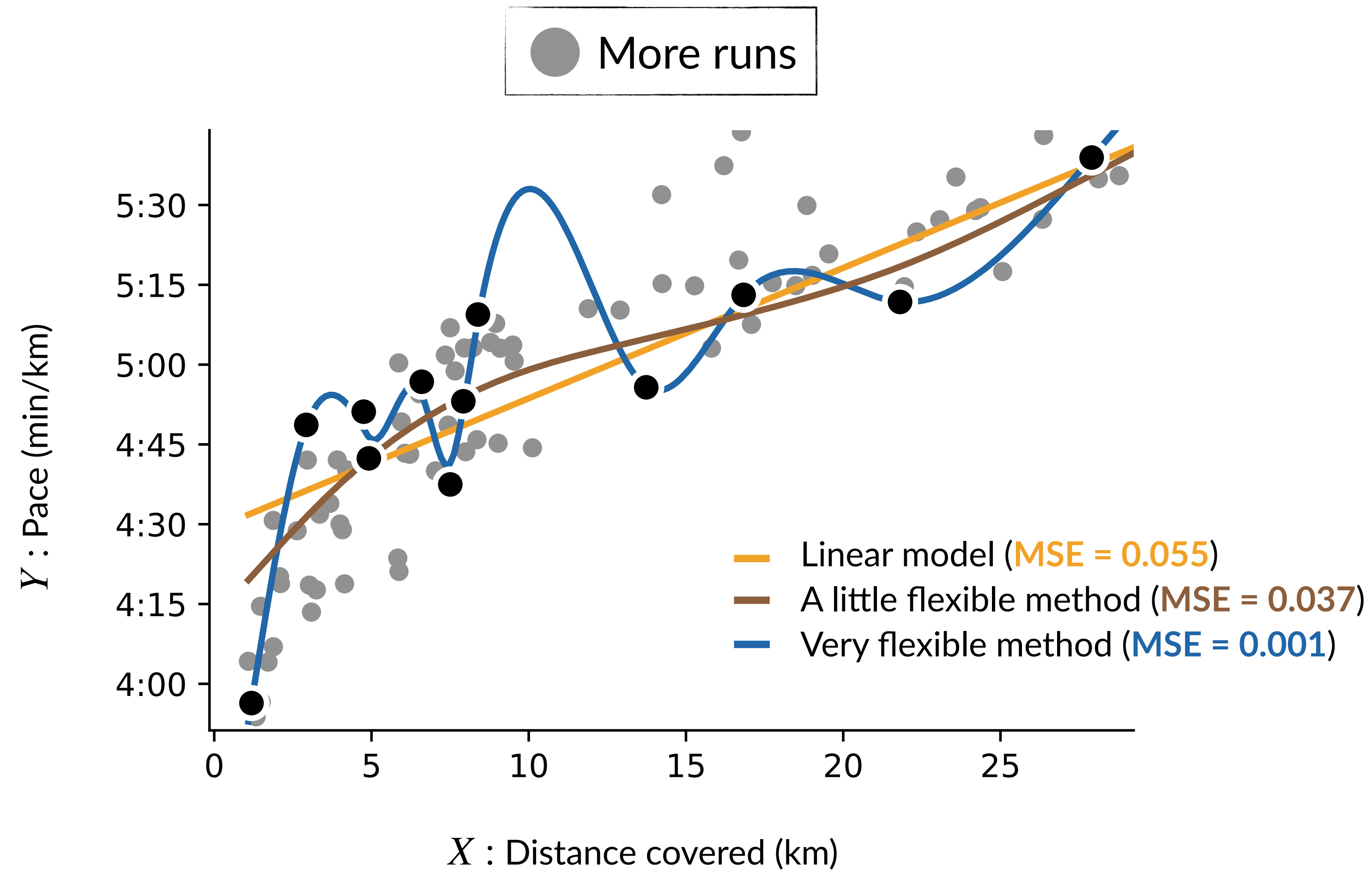
Modelling Pace

Which One Would You Choose?



Modelling Pace

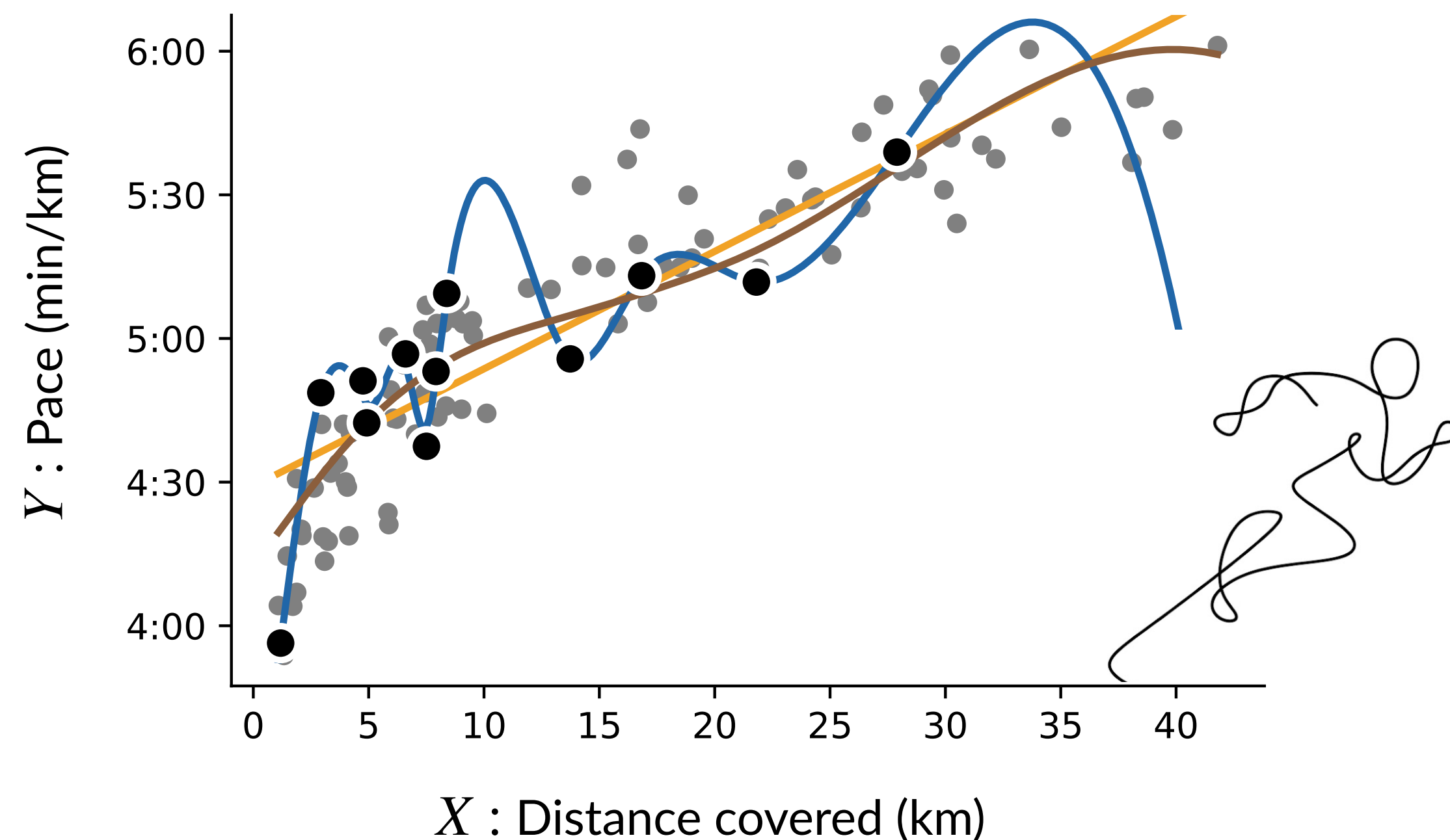
And now ?



Generalisation

Train vs Test Data

For illustration, this example has an unusually small training split, chosen here to make the effect visible. In practice, most of the data goes to training, with a smaller portion held out for testing



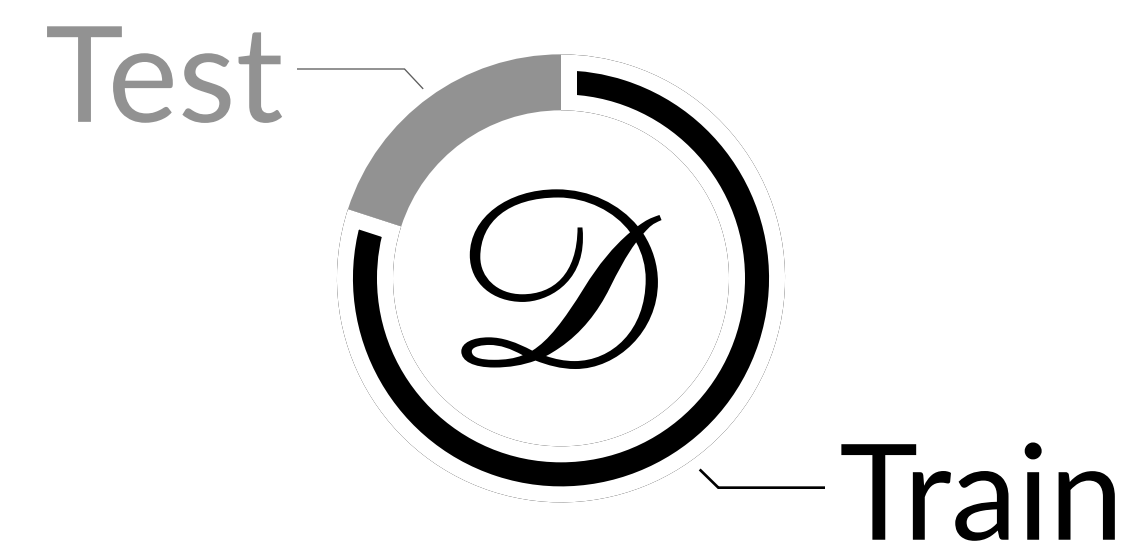
We split our dataset $\mathcal{D}$ into two disjoint parts:

$\mathcal{D}_{train}$: Training data

Used to fit $\hat{f}$ by Empirical Risk Minimisation.

$\mathcal{D}_{test}$: Test data

Held out, untouched during fitting. Used to check how $\hat{f}$ performs on data it's never seen



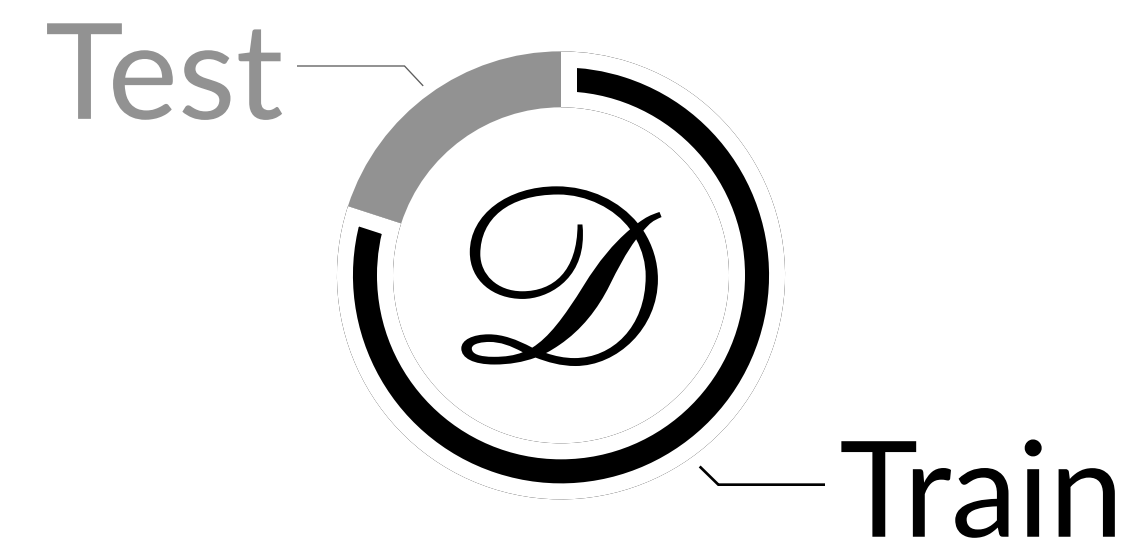
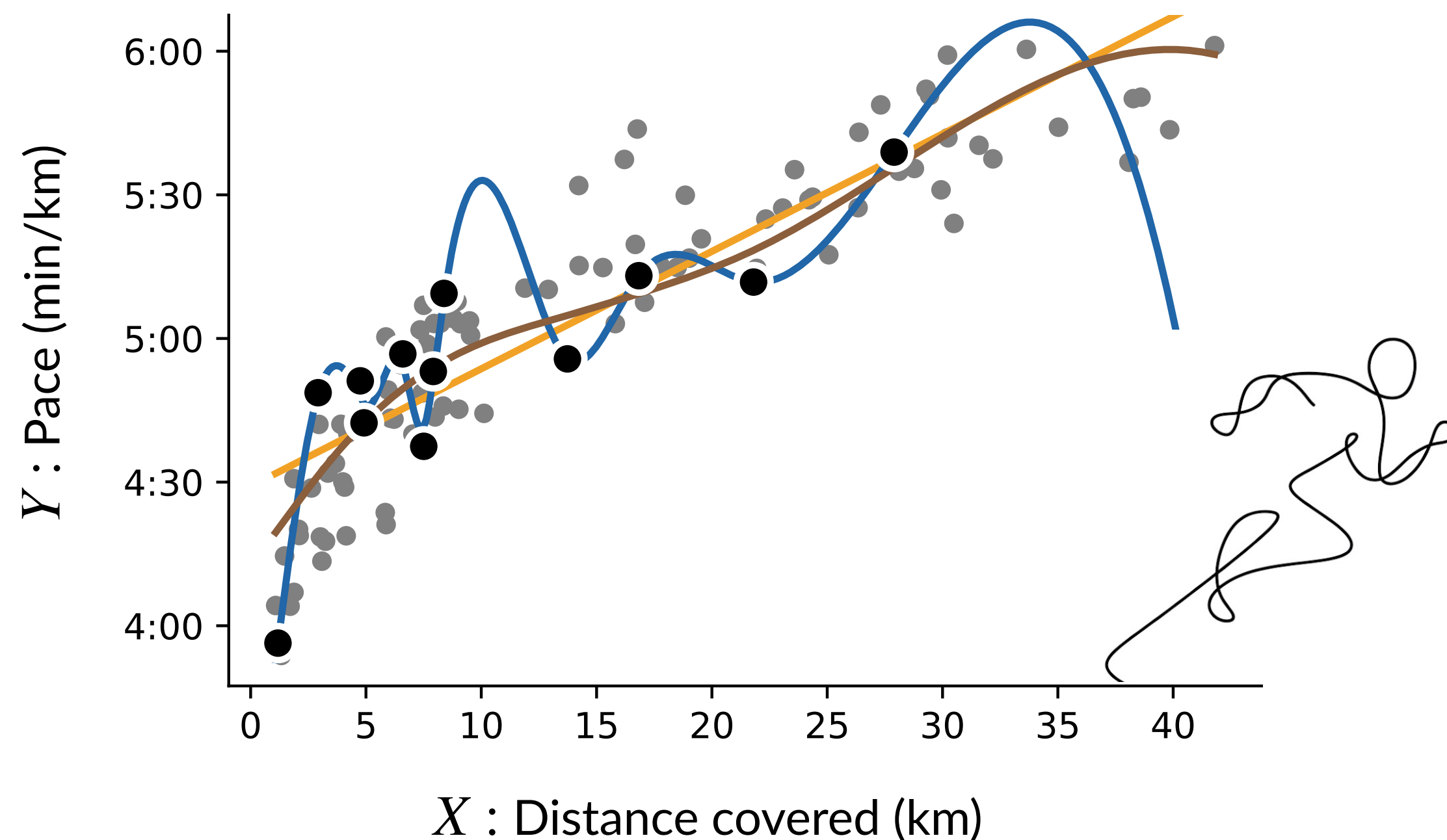
Generalisation

The Whole Point of Supervised Learning

We want $\hat{f}$ that performs well on new data.
Empirically, we seek the smallest possible **test error**

$$\text{MSE}_{\text{test}} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (y_i - \hat{f}(x_i))^2$$

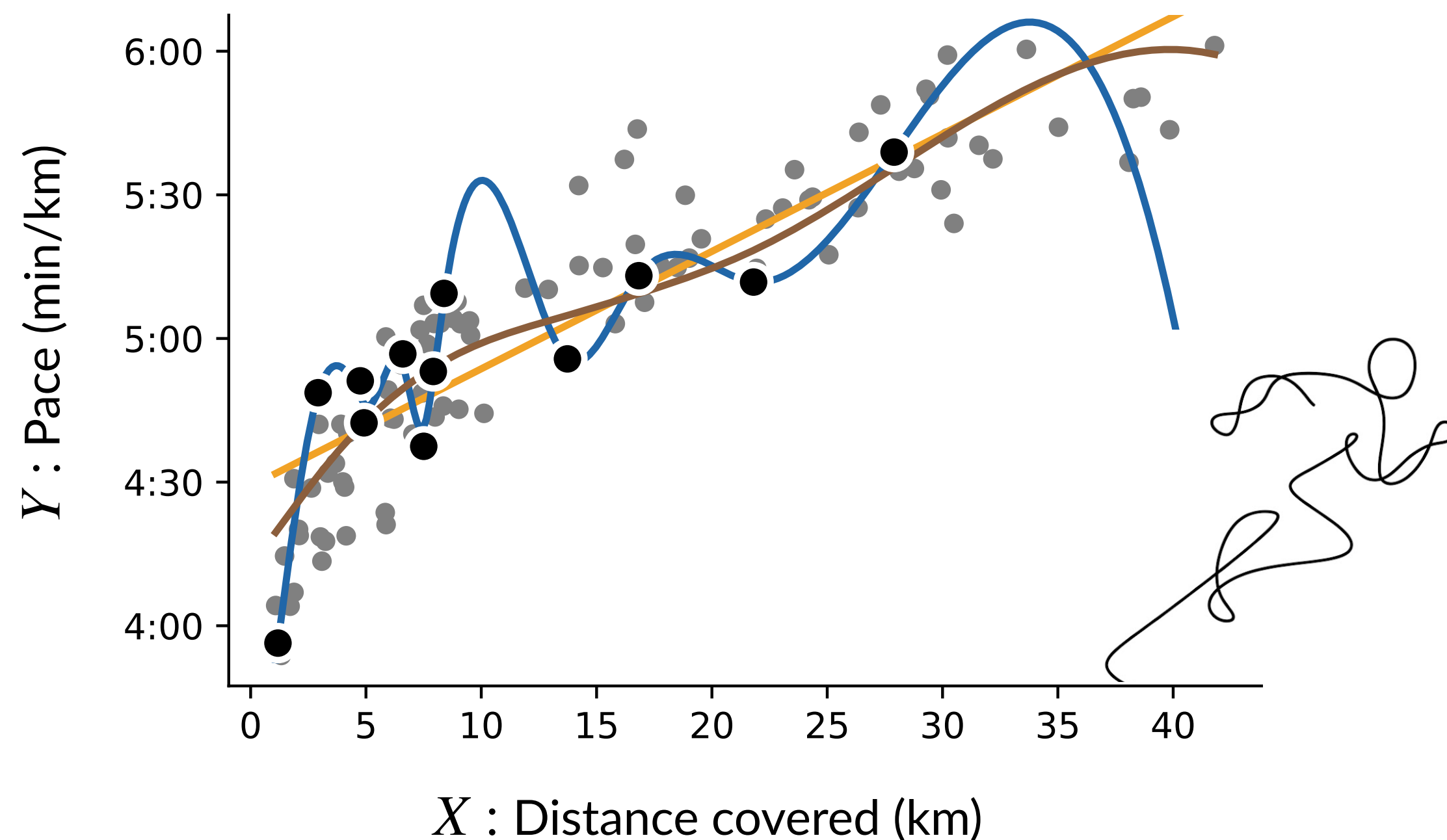
where $(x_i, y_i) \in \mathcal{D}_{\text{test}}$ are observations not used to train, or fit, $\hat{f}$



Generalisation

The Whole Point of Supervised Learning

The winner is... **A little flexible method** 🏆
Test MSE = 0.043, lowest of the three



We want $\hat{f}$ that performs well on new data.
Empirically, we seek the smallest possible **test error**

$$\text{MSE}_{\text{test}} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (y_i - \hat{f}(x_i))^2$$

where $(x_i, y_i) \in \mathcal{D}_{\text{test}}$ are observations not used to train, or fit, $\hat{f}$

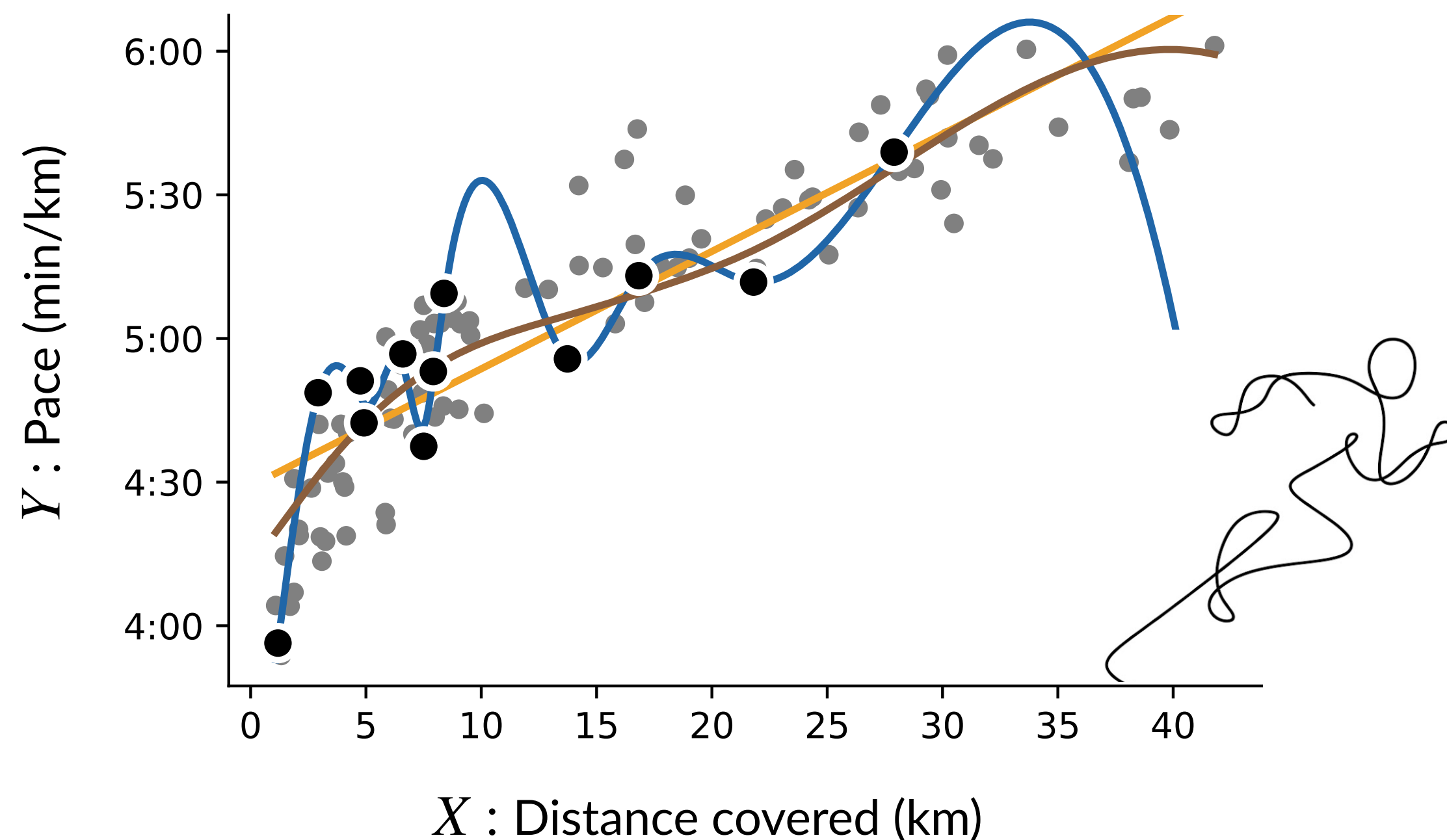
- Linear model (Train MSE = 0.055; Test MSE = 0.059)
 - A little flexible method (Train MSE = 0.037; Test MSE = 0.043)
 - Very flexible method (Train MSE = 0.001; Test MSE = 0.130)
- ⚠ There is no guarantee that a model with low $\text{MSE}_{\text{train}}$ will also have low MSE_{test}

Naming the Tension

How Much Flexibility is a Good Flexibility?

How does **test error** behave as **model flexibility** increases?

Two forces pulling in opposite directions: more flexibility lets fit the training data more closely, but it also makes more sensitive to whichever particular sample it happened to see

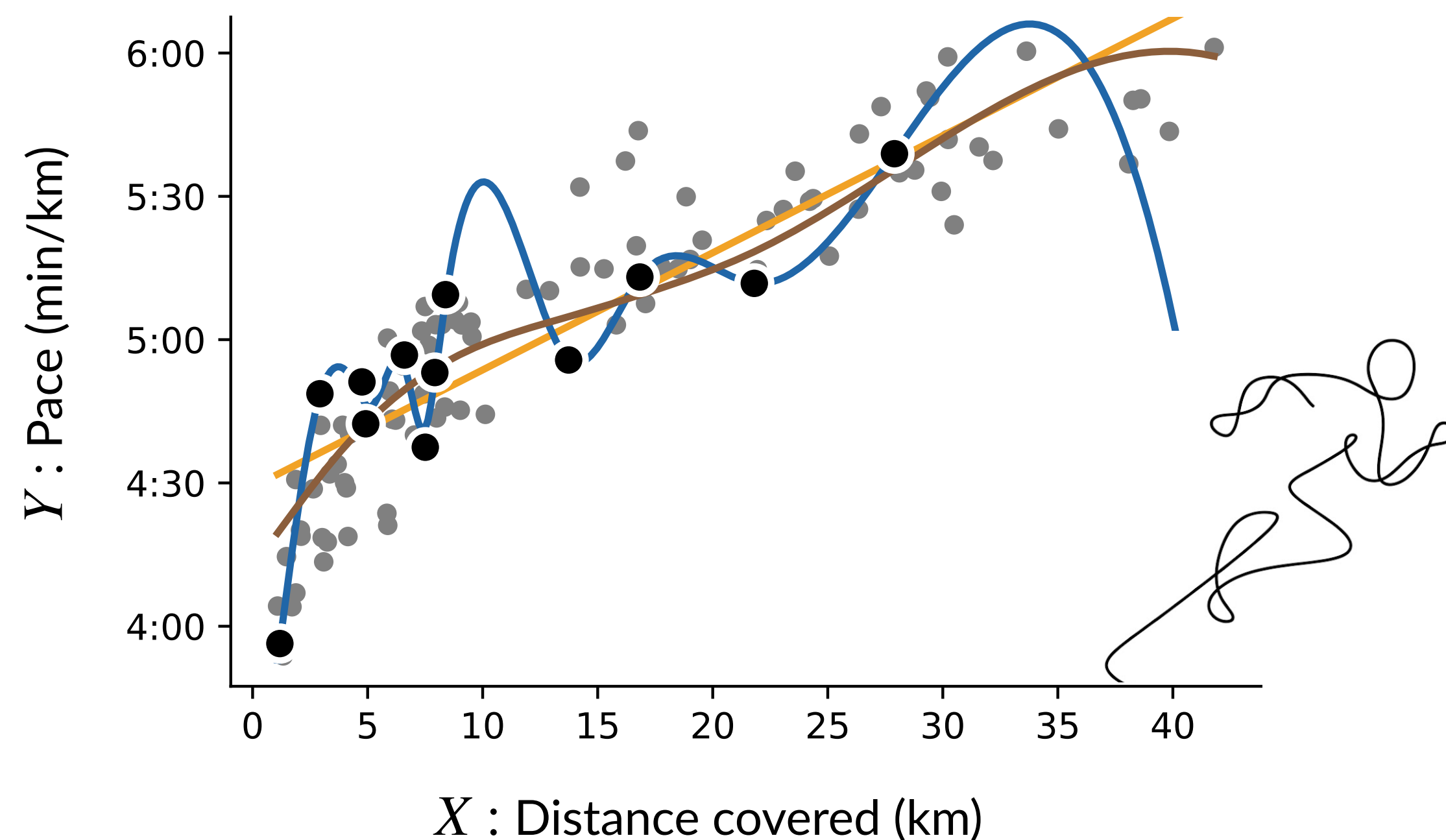


Naming the Tension

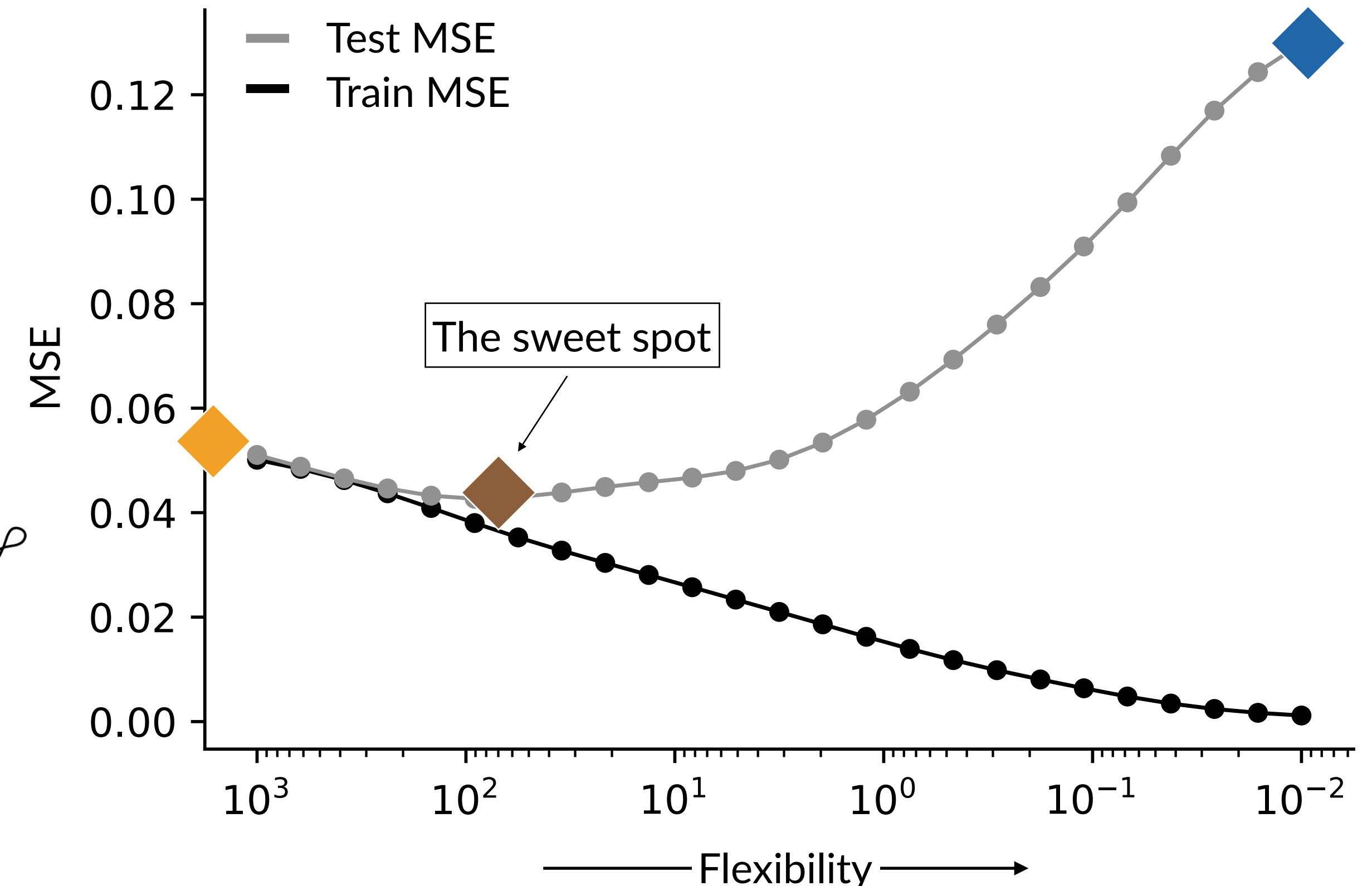
U-shaped Test MSE

How does **test error behave as model flexibility increases**?

Two forces pulling in opposite directions: more flexibility lets fit the training data more closely, but it also makes more sensitive to whichever particular sample it happened to see



We're using test error here to illustrate the shape of this trade-off—not as the sanctioned way to choose the right level of flexibility. That comes properly later in the course.



Naming the Tension

Underfitting vs Overfitting

How does **test error behave as model flexibility increases**?

Two forces pulling in opposite directions: more flexibility lets fit the training data more closely, but it also makes more sensitive to whichever particular sample it happened to see

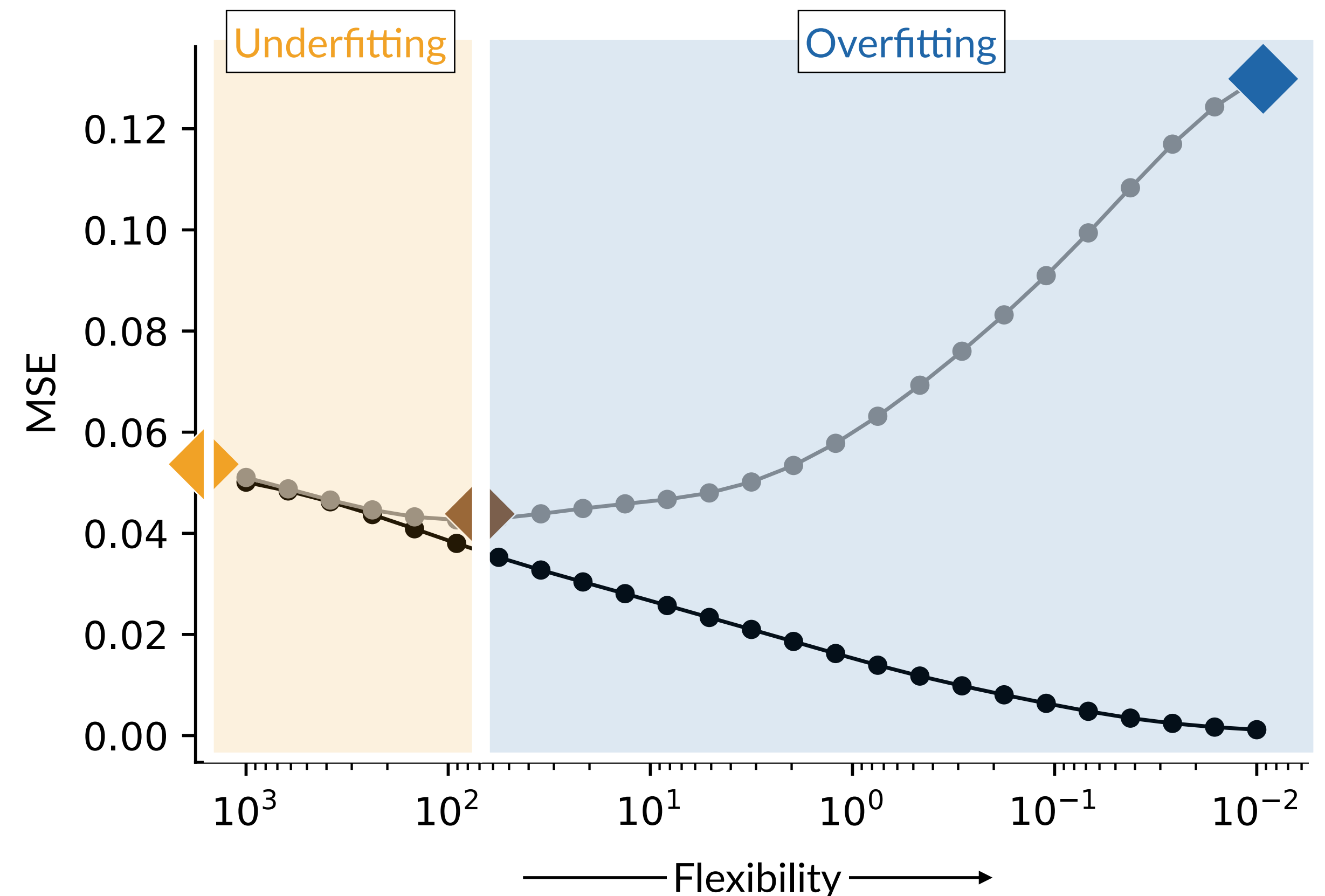
Underfitting

Flexibility too low, $\hat{f}$ **misses real structure**. Both training and test error are higher than they need to be

Overfitting

Flexibility too high, $\hat{f}$ **fits quirks specific to this training sample**. Training error keeps falling, but test error rises

We're using test error here to illustrate the shape of this trade-off—not as the sanctioned way to choose the right level of flexibility. That comes properly later in the course.

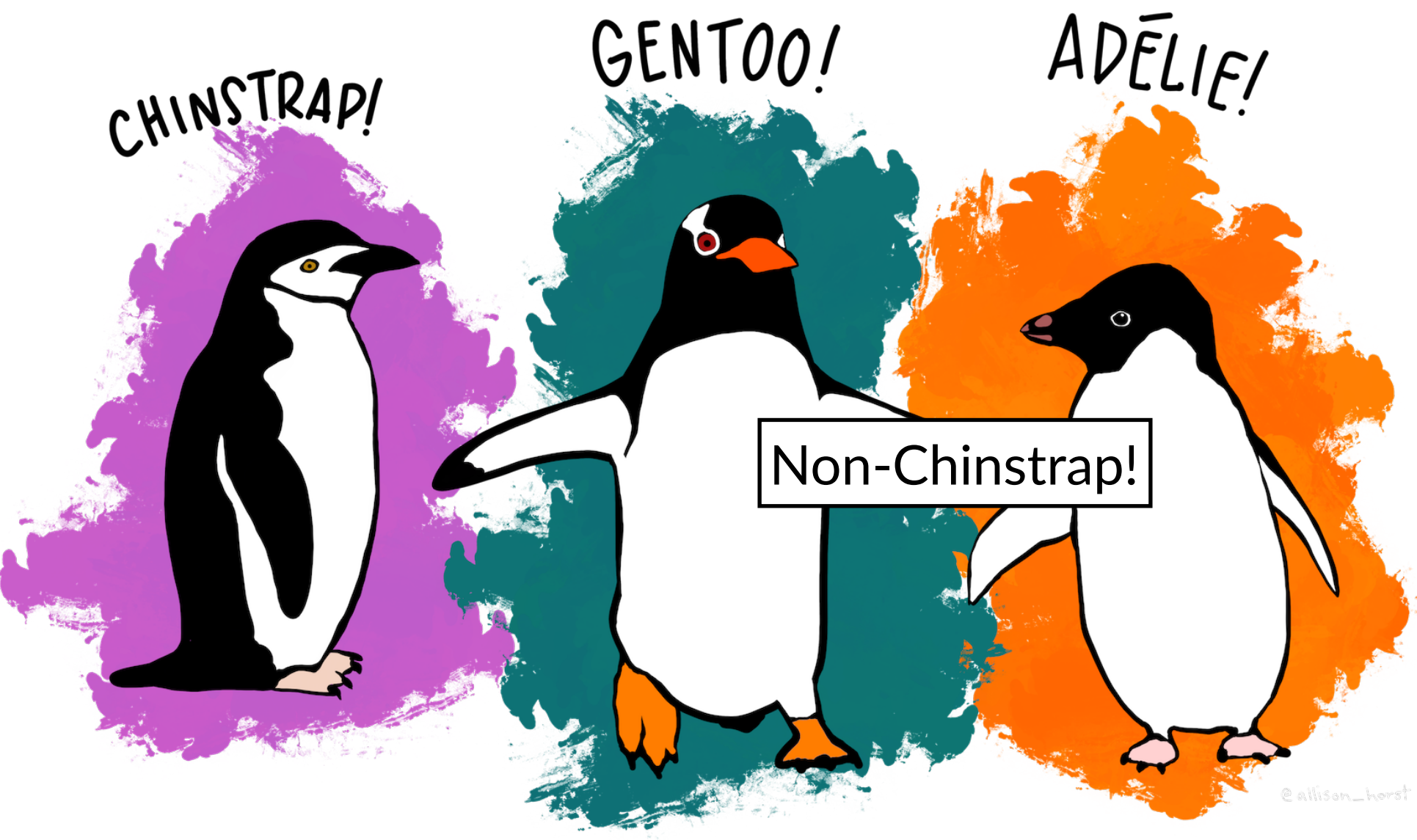
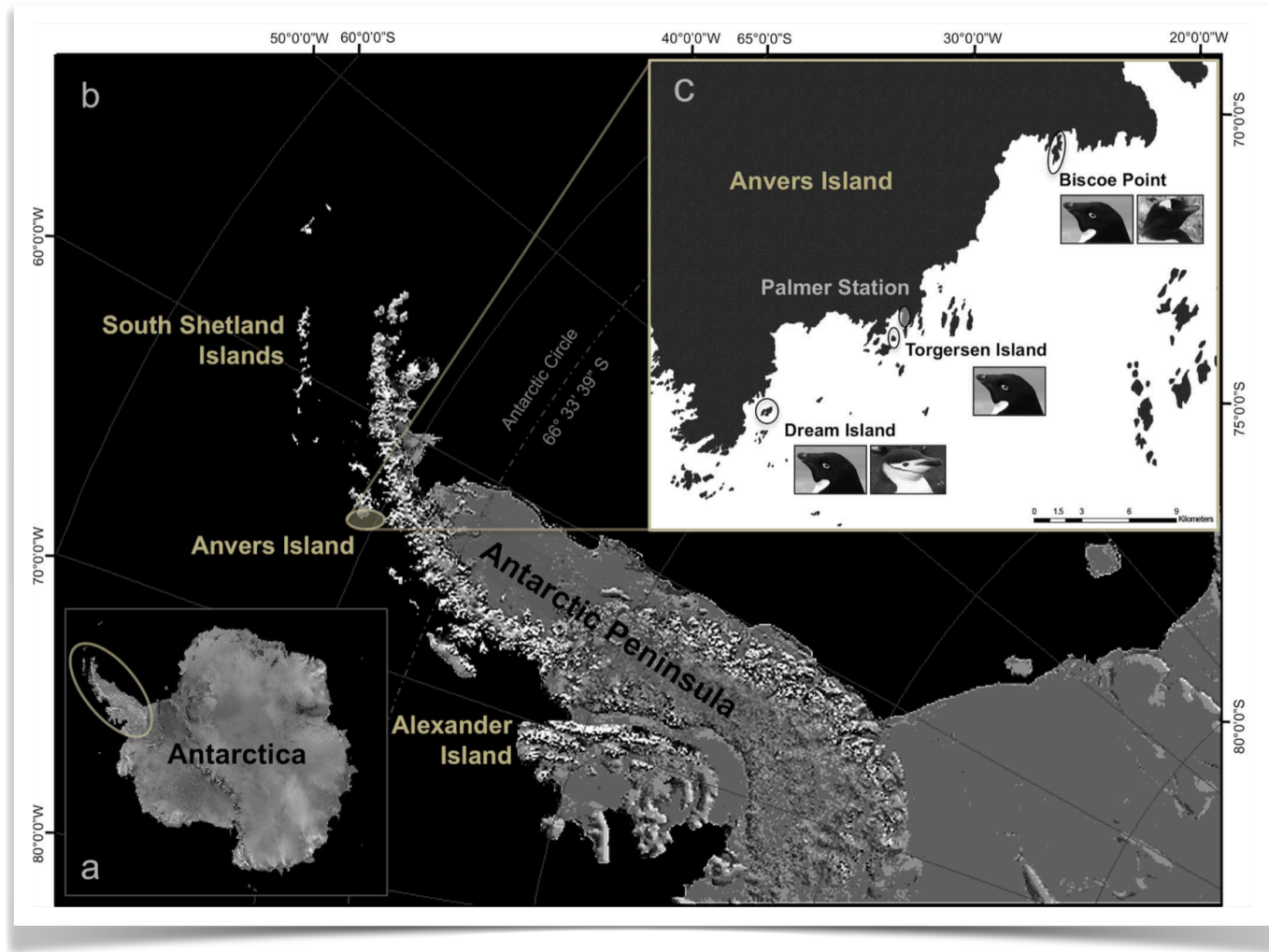
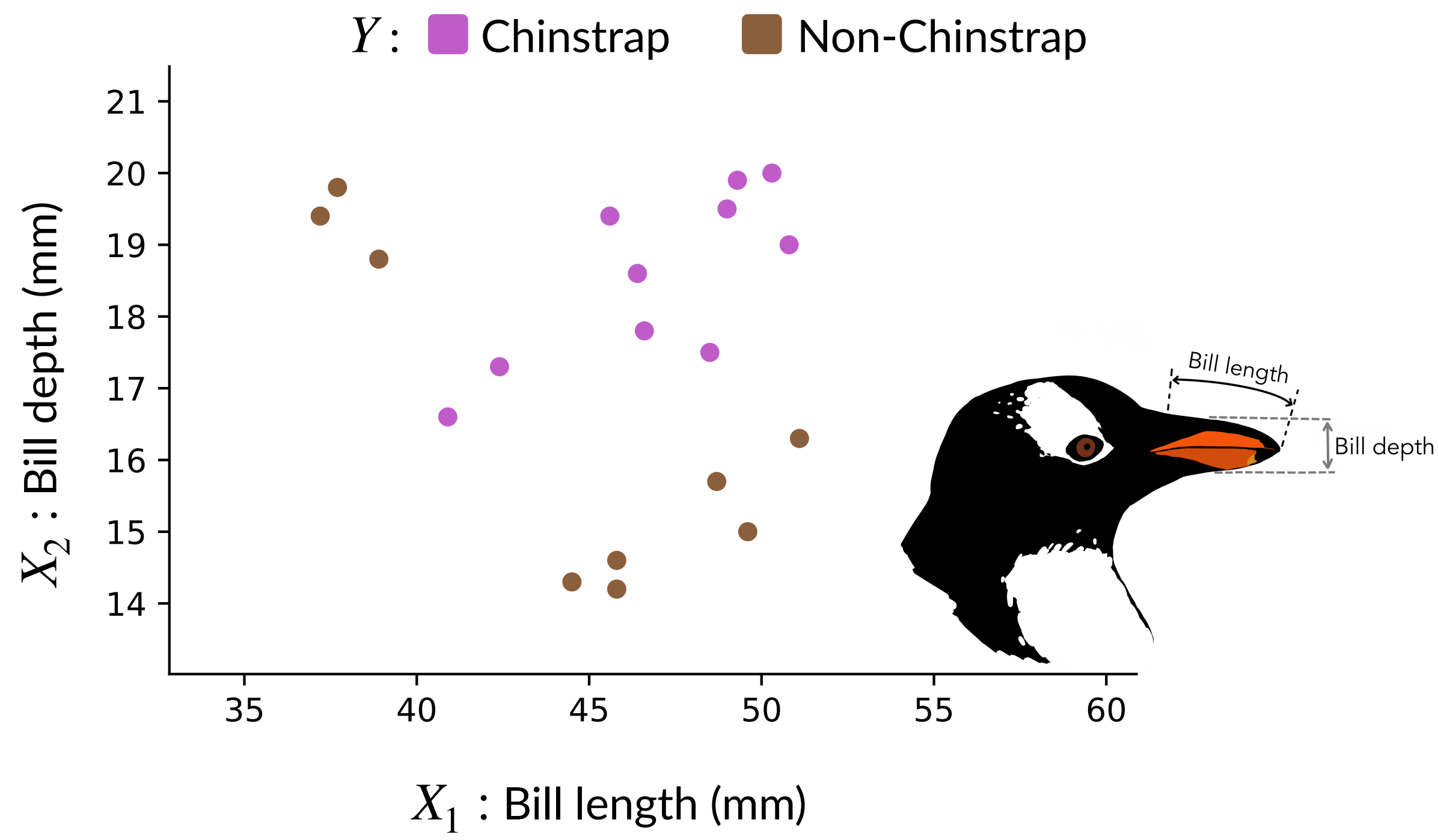


Classification

Back to Penguins

Classification /

Predicting whether a penguin is Chinstrap ($Y = 1$) or not $Y = 0$, from its bill length and depth



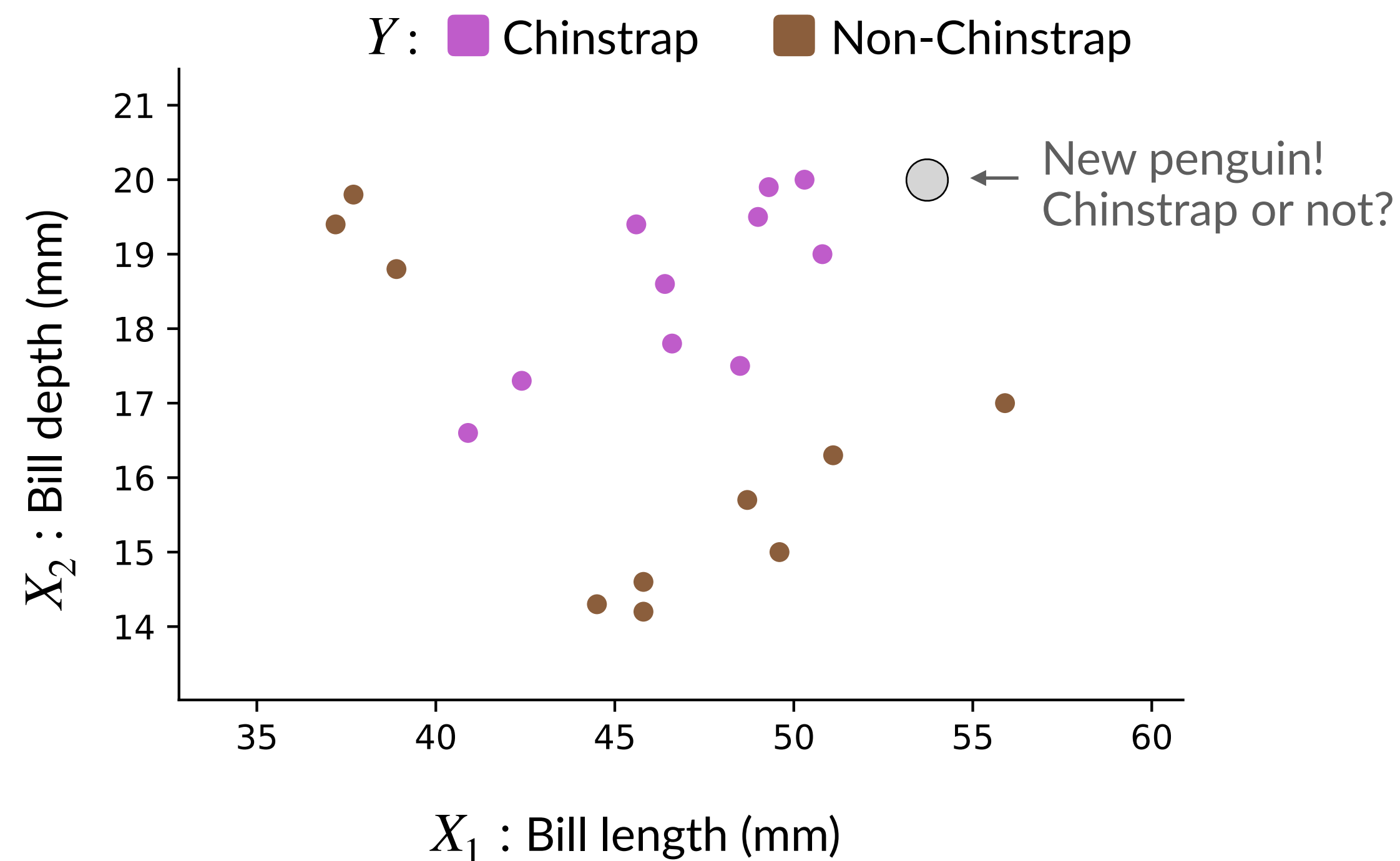
The cutest ever dataset: [palmerpenguins](#), available [here](#).
Original data in [Gorman+, 2014](#)

What Changes?

Target

Classification /

Predicting whether a penguin is Chinstrap ($Y = 1$) or not $Y = 0$, from its bill length and depth



What we predict /

Not a quantity, but **class probabilities**:

$$\hat{p}(\mathbf{X}) = \Pr(Y = 1 \mid \mathbf{X})$$

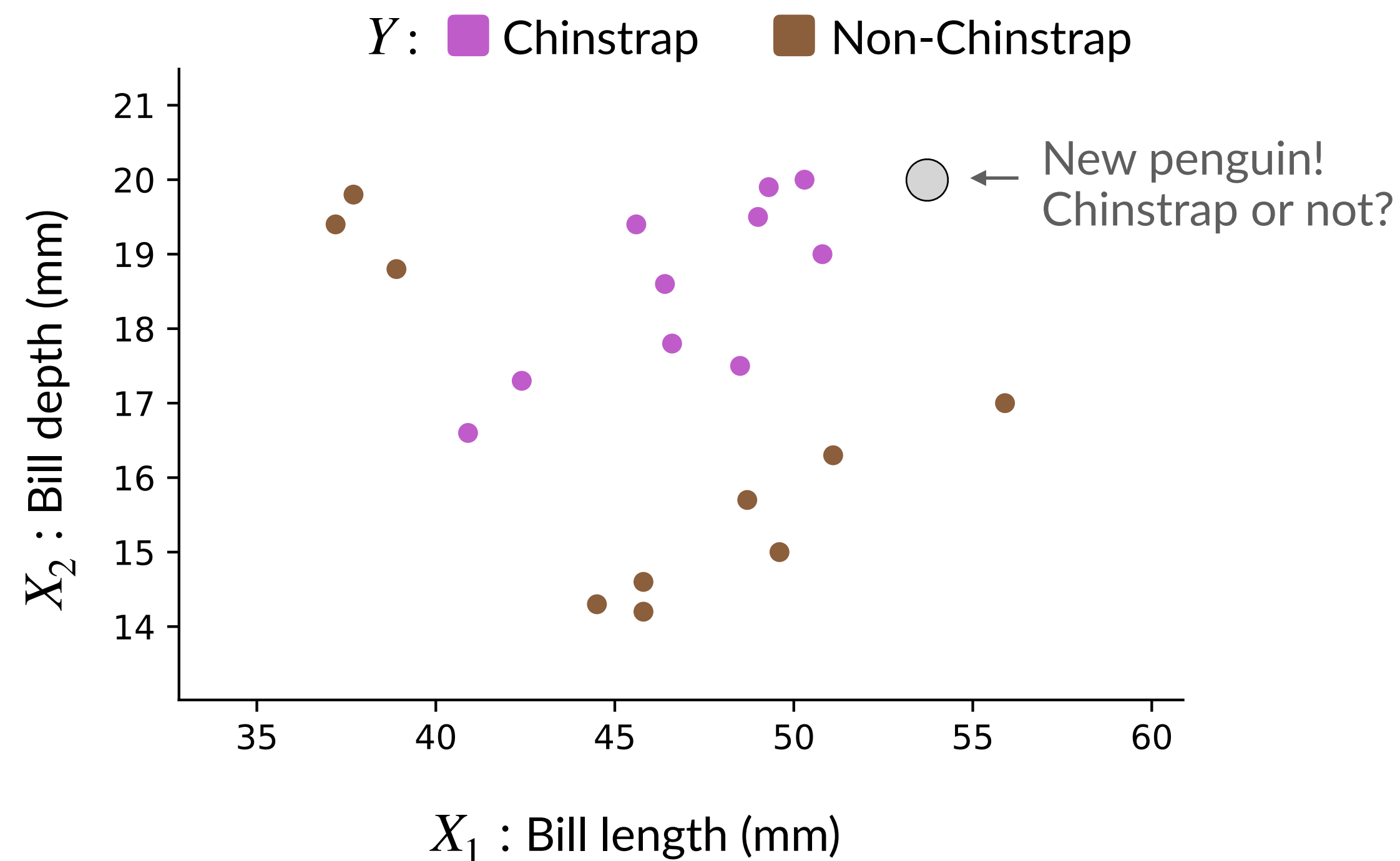
And we predict Chinstrap if $\hat{p}(\mathbf{X}) > 0.5$

What Changes?

Target, Error

Classification /

Predicting whether a penguin is Chinstrap ($Y = 1$) or not $Y = 0$, from its bill length and depth



What we predict /

Not a quantity, but **class probabilities**:

$$\hat{p}(\mathbf{X}) = \Pr(Y = 1 \mid \mathbf{X})$$

And we predict Chinstrap if $\hat{p}(\mathbf{X}) > 0.5$

How we measure error /

A natural measure is the **error rate**: the fraction of predictions that are wrong

$$\text{Error rate} = \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}_i)$$

Indicator function

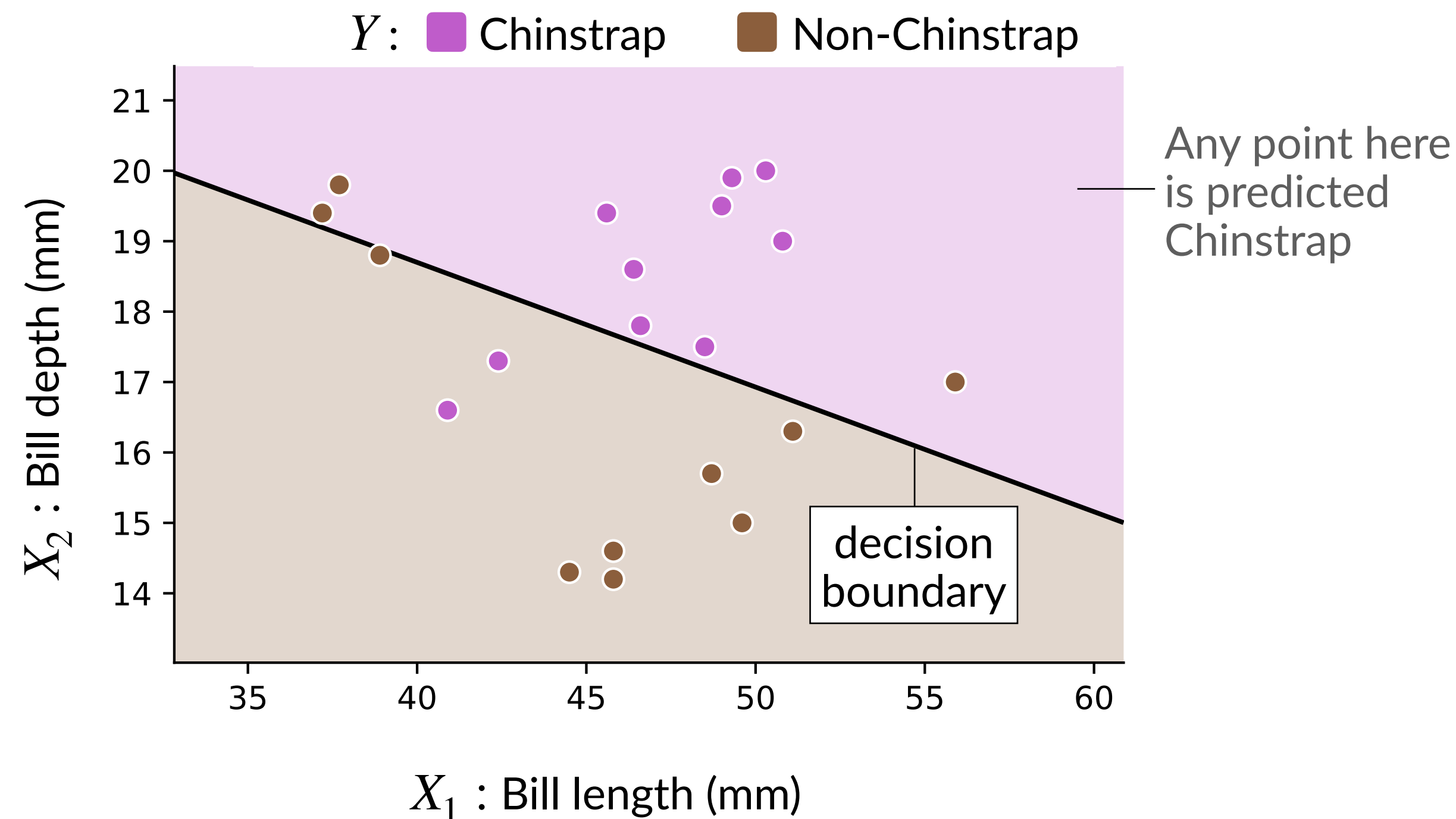
It is 1 if the prediction is wrong, and 0 if it's right

Modelling Penguins

A Linear Attempt

Classification /

Predicting whether a penguin is Chinstrap ($Y = 1$) or not $Y = 0$, from its bill length and depth



Historical accident,
this is classification!

Logistic Regression

The **simplest** non-trivial **parametric** choice:
assume the log-odds of $Y = 1$ depend linearly $\mathbf{X}$:

$$\hat{p}(X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2)}}$$

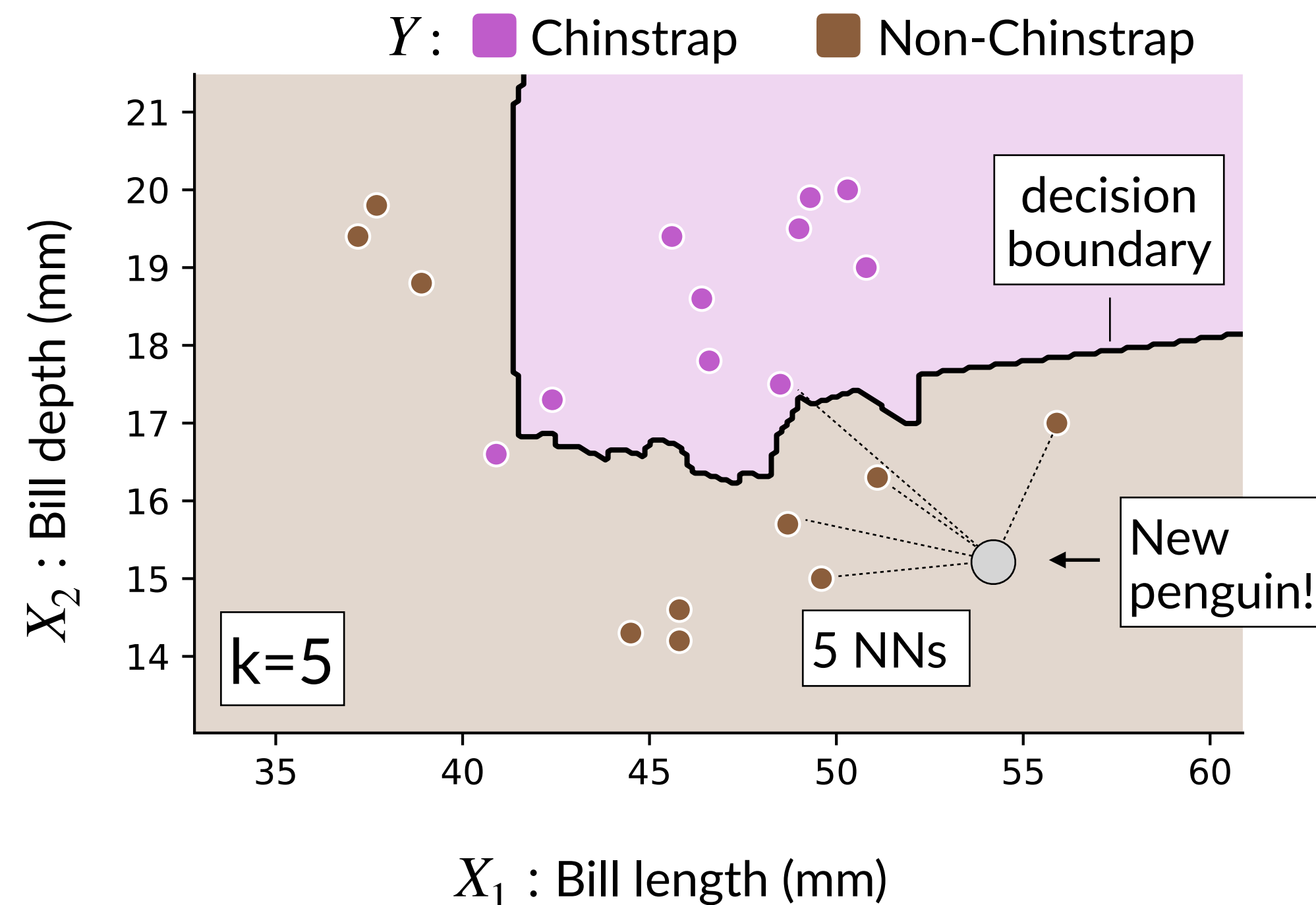
Same trade-off as with the linear model for regression: **simple, interpretable and cheap**, but **rigid**—the decision boundary can only ever be a straight line

Modelling Penguins

A Flexible Attempt

Classification /

Predicting whether a penguin is Chinstrap ($Y = 1$) or not $Y = 0$, from its bill length and depth



k-Nearest Neighbours

A **non-parametric** method: classify each point by a vote among its k nearest neighbours. The boundary follows the data directly. How local the vote is controlled by a **hyperparameter**, k .

$$\hat{p}(X) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(X)} I(y_i = 1)$$

Same trade-off as with the splines for regression: **flexible and data-driven**, but that **flexibility has to be tuned** and it is **harder to interpret**

This is also a **local method**—it relies on nearby points to make a decision. In high dimensions, these models struggle: it is the « curse of dimensionality »—we'll come back to this

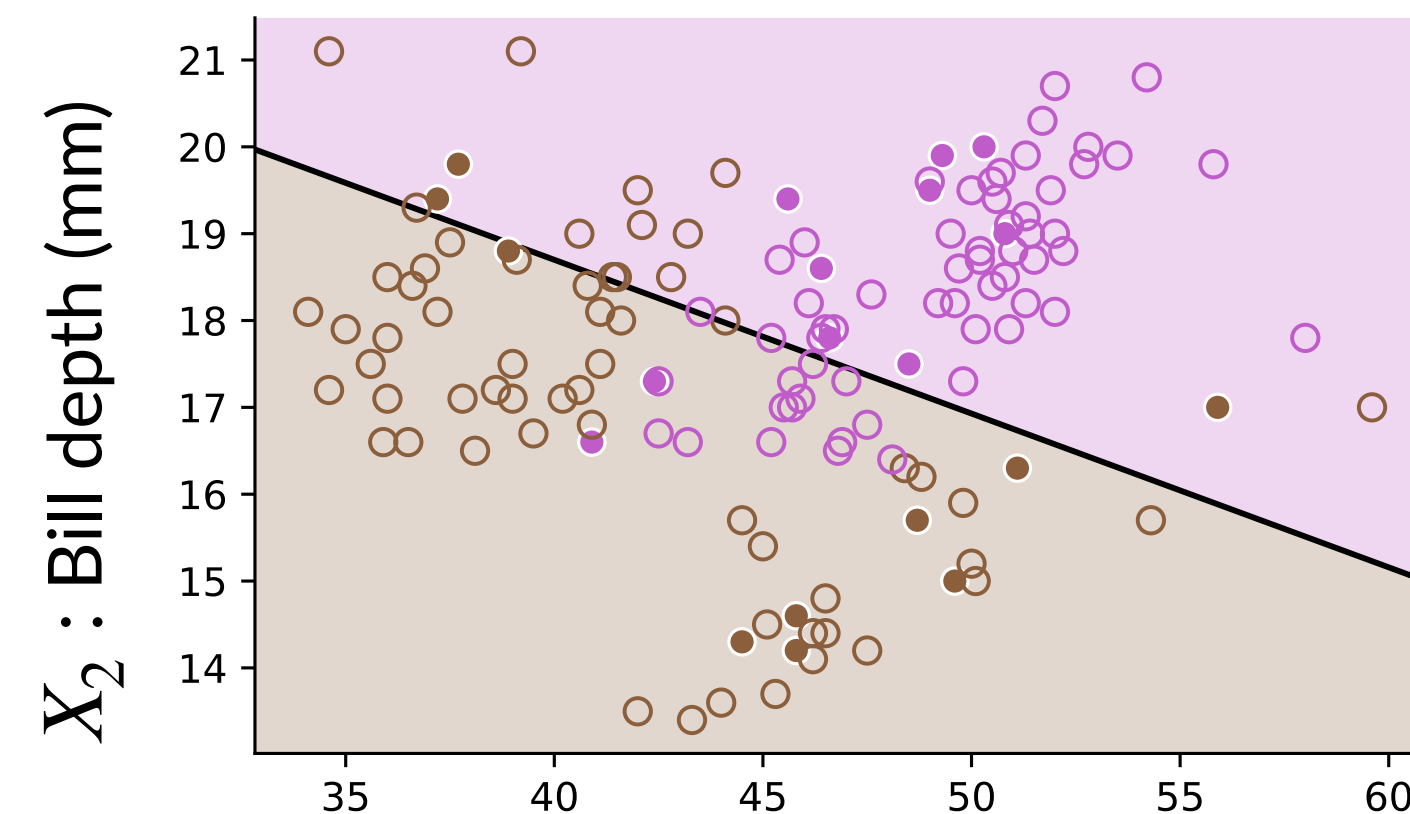
Modelling Penguins

Which One Would You Choose? Well, Now You Know



Train ER = 0.25 ; Test ER = 0.23

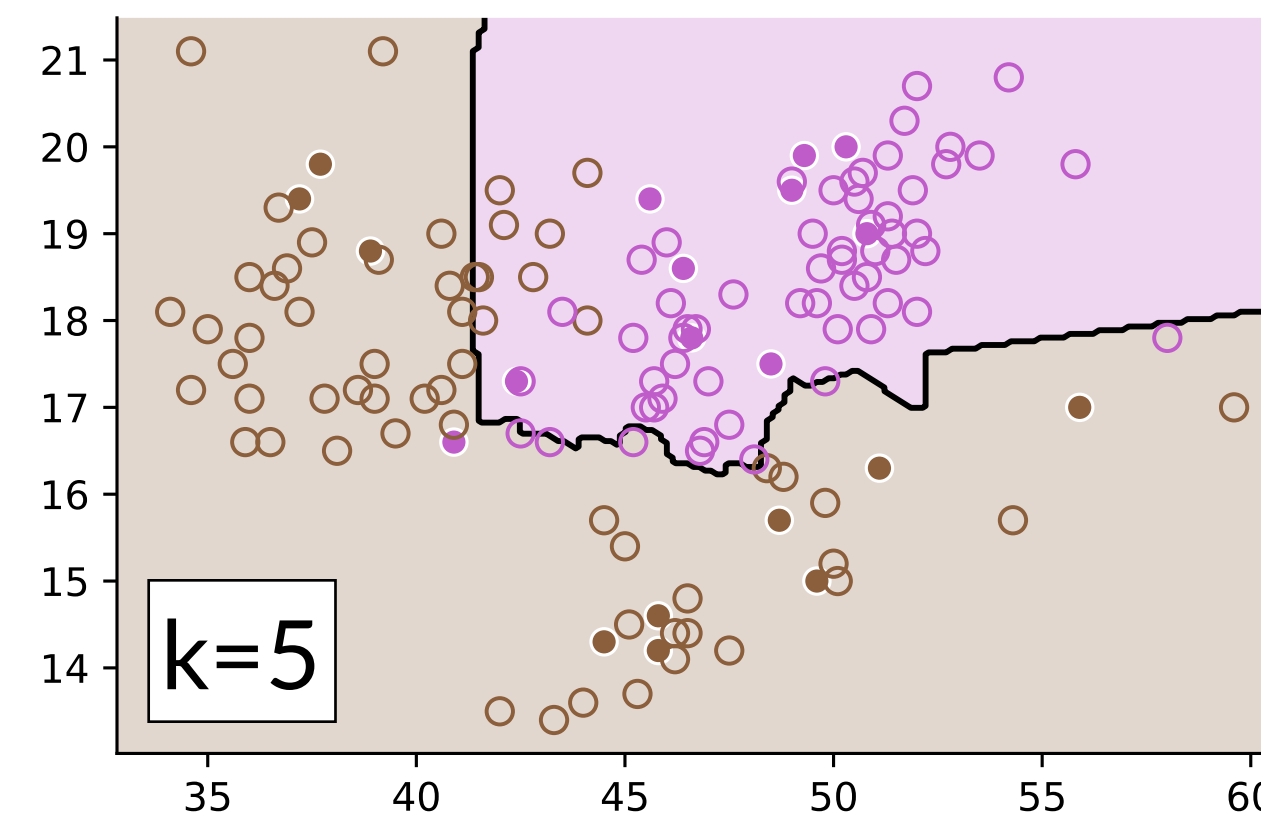
Logistic Regression



X_1 : Bill length (mm)

Train ER = 0.05 ; Test ER = 0.10

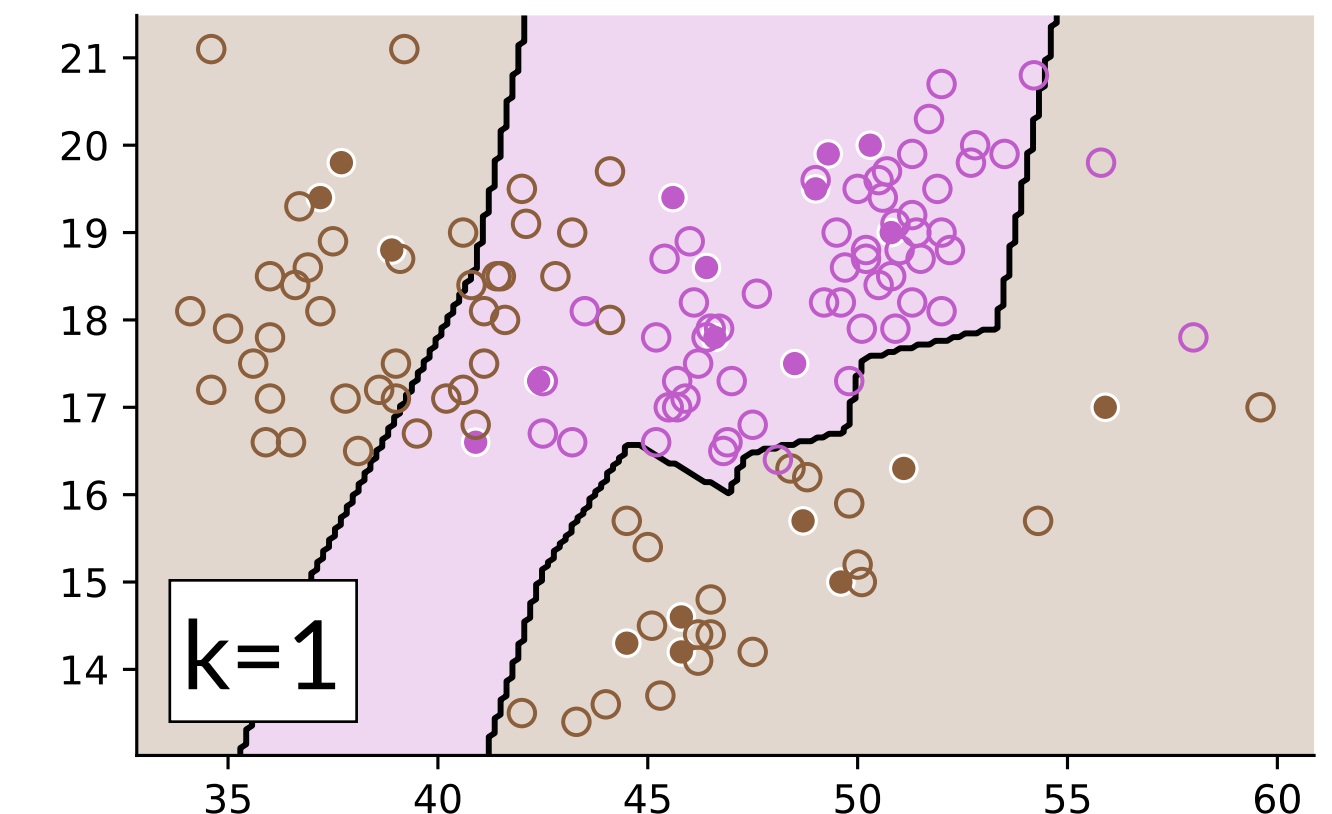
k-NNs



X_1 : Bill length (mm)

Train ER = 0.00, Test ER = 0.016

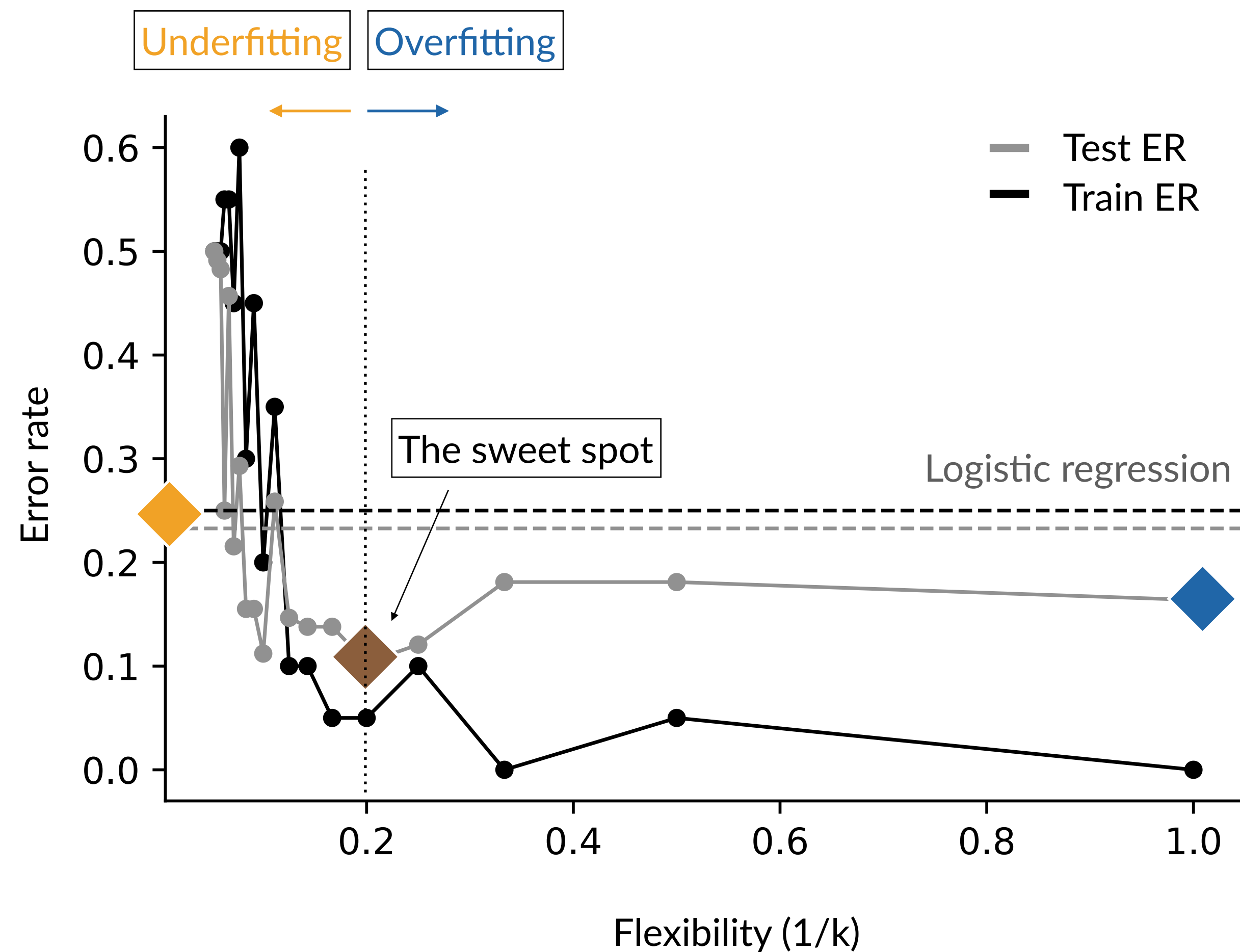
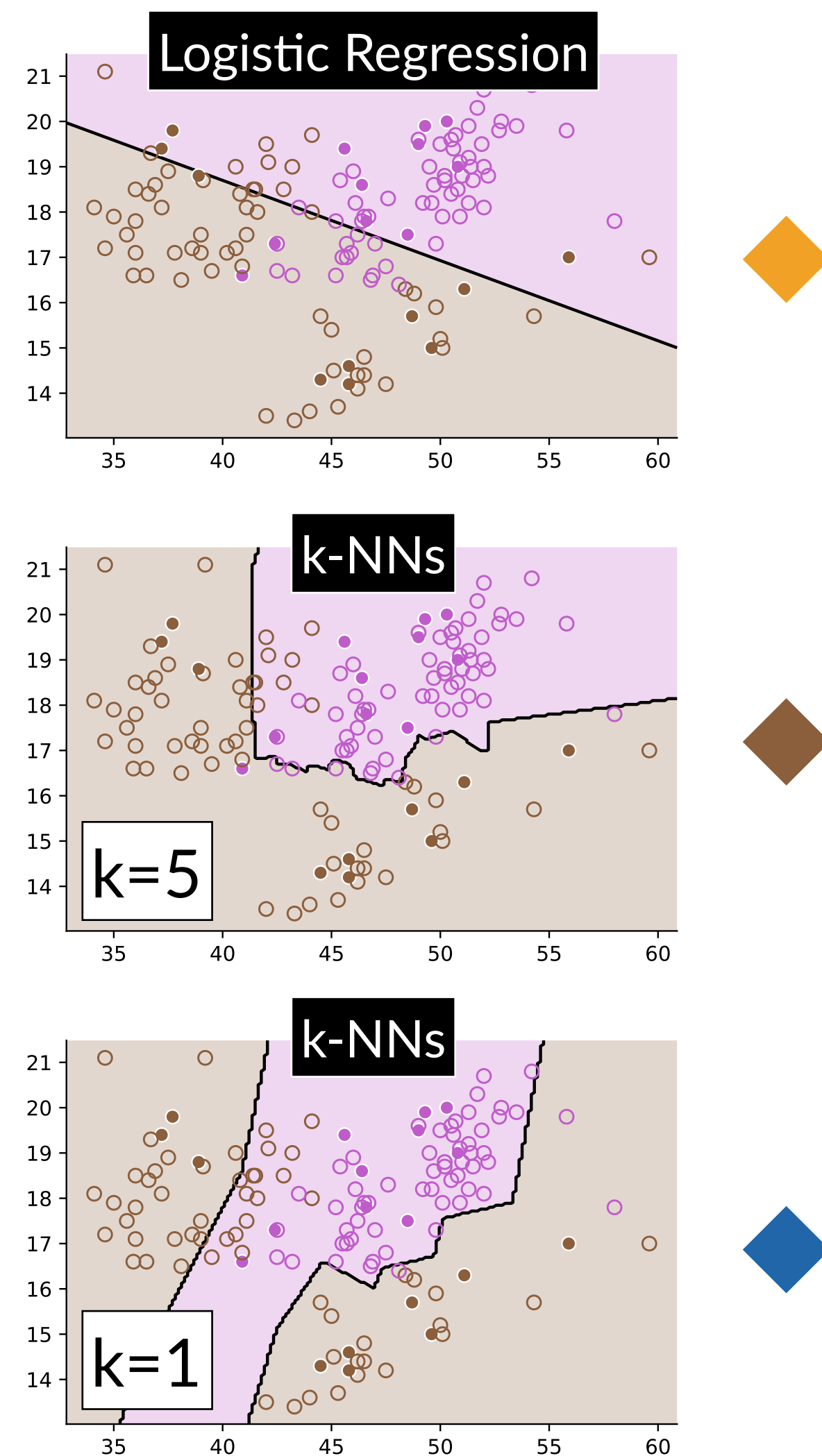
k-NNs



X_1 : Bill length (mm)

● Train data ○ Test data

Naming The Tension Again



Lecture Report

2 : Statistical Learning



Key concepts:

- ✓ The statistical learning framework
- ✓ Supervised vs. unsupervised learning
- ✓ Regression vs. classification
- ✓ Prediction vs. inference
- ✓ Parametric vs. non-parametric methods
- ✓ Training error vs. test error
- ✓ Underfitting vs. overfitting

Bibliography:

- ◎ ISL, C2

To go further:

- ◎ ESL, C1

Next week:

Linear Regression